



CDA 650 – Special Topics

Experiential Projects in Artificial Intelligence and Data Science

Spring 2026

AI-Powered Vibration Fault Diagnosis for Rotating Machinery

Authors

Manimegalai Mamallan

Saakketh Gouti

Sandhya Mishra

Faculty Advisor

Prof. Adithya Narayanan

Prof. Saverio Marrazzo

Industry Advisor

Mr. John Biondolillo

Machinery Monitoring Systems LLC

Abstract

Reliable and early detection of faults in rotating machinery is essential for maintaining industrial safety, minimizing unplanned downtime, and enabling effective predictive maintenance. In many industrial settings, maintenance remains reactive, creating a need for models that can identify fault signatures much earlier from subtle patterns in vibration behavior, and that can also estimate the severity of those faults so that maintenance teams can prioritize intervention before failure. In this work, we evaluate a range of classical and deep learning time-series classification approaches including Random Forest with hand-engineered features, 1D Convolutional Neural Networks, 2D CNNs operating on spectrogram representations, hybrid 1D CNN with LSTM, ResNet-based architectures, MINIROCKET, and Multi-Scale CNNs for classifying five machine conditions: normal, bearing fault, unbalance, misalignment, and mechanical looseness, and for estimating fault severity at three levels, Low, Medium, and High, for each applicable fault class.

Building on insights from these evaluations, we develop a two-stage deep learning system in which a multi-label fault classifier produces independent per-class probabilities through a sigmoid output, and conditional per-fault severity classifiers produce softmax distributions over severity levels for each detected fault. The system processes triaxial vibration data across approximately 50K samples spanning four fault classes, three severity levels, and three rotational speeds, with features extracted in both the time and frequency domains and stratified jointly across all three dimensions during training. To further enhance feature richness, the Multi-Scale CNN incorporates parallel convolutional kernels of varying receptive fields, enabling the model to learn both narrow harmonic peaks such as unbalance at $1\times$ rotation and broader spectral envelopes such as mechanical looseness, while MINIROCKET captures impulse and periodicity patterns through ten thousand random convolutional features followed by a linear ridge classifier.

This combined time-domain, frequency-domain, and multi-resolution representation reduces reliance on handcrafted spectral heuristics and improves generalization to diverse fault patterns. Across all experiments, the proposed system achieves near-perfect accuracy on all five fault classes and high accuracy on per-fault severity classification, with sub-50 millisecond inference latency per triplet on commodity CPU hardware, supporting both batch file analysis and real-time streaming deployment. These results demonstrate the effectiveness of integrated multi-resolution vibration modeling combined with a two-stage classification design for joint fault detection and severity

estimation, and establish a strong foundation for future research in vibration-based machine health diagnostics, including multi-fault overlapping signatures, speed generalization beyond the trained range, and domain transfer to industrial-scale machinery.

Keywords: Vibration analysis, fault diagnosis, severity prediction, MINIROCKET, multi-scale CNN, multi-label classification, FFT, predictive maintenance.

Contents

Abstract	2
List of Figures	9
List of Tables	9
1 Introduction	1
1.1 Background and Motivation	1
1.2 Problem Statement	3
1.3 Objectives	4
1.4 Scope of the Project	5
2 Experimental Setup	6
2.1 Rotating Machinery Test Rig	6
2.2 Data Acquisition	7
2.3 Four Different Setups	7
2.3.1 First Setup	7
2.3.2 Second Setup	8
2.3.3 Third Setup	9
2.3.4 Fourth Setup (Working Setup)	11
2.4 Operating Conditions: Fault Simulation and Severity Simulation	11
2.4.1 Normal Operating Condition	12
2.4.2 Unbalance	13
2.4.3 Mechanical Misalignment	14
2.4.4 Mechanical Looseness Condition	15
2.4.5 Bearing Fault Condition	16
2.5 Collected Raw Data Format and Structure	17
2.6 Data Ingestion and Dataset Construction	18
3 Dataset Preparation and Feature Engineering	20
3.1 Dataset Overview and Notation	20
3.2 Frequency-Domain Feature Generation	21

3.3	Train-Validation-Test Split Protocol	21
3.4	Feature Scaling (Standardization)	22
4	Models and Architectures	24
4.1	Model Input-Output Formulation	24
4.2	Motivation for Dual-Domain Inputs	25
4.3	High-Level Model Overview	26
4.4	Baseline Models	27
4.4.1	1D Convolutional Neural Network	27
4.4.2	2D Convolutional Neural Network on Spectrogram Representations	29
4.4.3	Multi-Scale Convolutional Neural Network	30
4.4.4	MINIROCKET with Ridge Classifier	31
5	Training Setup and Evaluation Metrics	34
5.1	Training Configuration	34
5.1.1	Loss Functions	34
5.1.2	Optimizer and Learning Rate	35
5.1.3	Batch Size and Epochs	36
5.1.4	Training Callbacks	36
5.1.5	Reproducibility	36
5.2	Evaluation Metrics	36
5.2.1	Accuracy	37
5.2.2	Macro F1-Score	37
5.2.3	Confusion Matrix	37
5.2.4	Per-Class Precision, Recall, and F1	38
5.2.5	AUC-ROC	38
5.3	Experimental Protocol and Reproducibility	38
5.3.1	Code Availability	38
5.3.2	Hardware and Software Environment	39
5.3.3	Random Seeds	39
5.3.4	Inference-Time Consistency	39
5.3.5	Implementation Details	40
6	Results and Discussion	41
6.1	Experimental Setup Progression	41
6.2	Overall Model Performance on the Final Setup	42
6.3	Severity Classification Results	43
6.4	Confusion Matrix Analysis	44
6.4.1	1D CNN	44

6.4.2	Multi-Scale CNN	44
6.4.3	MINIROCKET	44
6.4.4	2D CNN	44
6.4.5	1D CNN with LSTM (Proof of Concept)	46
6.5	Discussion and Interpretation	47
6.5.1	Why Performance Is High on Setup 4	47
6.5.2	Multi-Scale Versus Single-Scale Architectures	48
6.5.3	Multi-Fault Detection and Architectural Generalization	48
6.5.4	Computational Considerations	49
6.5.5	Practical Implications for Predictive Maintenance	49
7	Inference and Deployment	51
7.1	System Overview and Project Structure	51
7.2	Backend Inference Service	53
7.2.1	Model Registry and the Predictor Contract	53
7.2.2	Three-Model Ensemble and Aggregation Logic	54
7.2.3	Severity Inference and Cascaded Decisions	55
7.3	Frontend Application (Streamlit)	55
7.3.1	Dashboard Layout	55
7.3.2	Live Streaming Mode	58
7.4	Visualization and Signal Metrics	59
7.4.1	Waveform and Spectrum Plots	59
7.4.2	Confidence and Severity Indicators	59
7.4.3	Per-Band Energy and Detection Rate	60
7.5	Deployment Considerations	60
7.5.1	Packaging and Cross-Platform Compatibility	60
7.5.2	Inference Latency and Throughput	61
7.5.3	Reproducibility through Pinned Dependencies	61
7.5.4	Robustness and Graceful Degradation	62
7.5.5	Future Packaging Extensions	62
8	Conclusion and Future Work	64
8.1	Summary of Contributions	64
8.2	Experimental Limitations	65
8.2.1	Single-Fault Training Data	65
8.2.2	Bounded Speed Range	66
8.2.3	Controlled Fault Severity	66
8.2.4	Domain Specificity	66
8.2.5	Self-Supervised Pretraining Considerations	66

8.3	Future Work	67
8.3.1	Data Collection on Industrial-Scale Machinery	67
8.3.2	Multi-Fault and Compound Fault Modeling	67
8.3.3	High-Speed and Variable-Speed Operation	67
8.3.4	Progressive Severity and Fault Evolution Modelling	68
8.3.5	Live Streaming and Continuous Monitoring	68
8.3.6	Operator-Facing UX Refinements	68
8.4	Concluding Remarks	68

List of Figures

2.1	First Setup: rotor on the left of sensor which is mounted on ball bearing.	8
2.2	Second Setup: rotor on the right of ball bearing and sensor at the end mounted on sleeve bearing.	9
2.3	Third Setup: rotor on the left of ball bearing and every part is equally distanced. . .	10
2.4	Fourth Setup: rotor on the right of sleeve bearing and every component (motor, sleeve bearing, rotor, and sensor at the end) is equally distanced.	11
2.5	Normal Condition: Fourth setup in normal condition where data was collected for 2000, 2500, and 3000 RPM.	12
2.6	Unbalanced condition: shows the unbalanced condition for high severity (6 screws) simulated in the lab. Low (2 screws) and medium (4 screws) configurations are also used to collect data for all severity levels, keeping everything else in the setup the same except for the number of screws.	13
2.7	Mechanical misalignment: fourth setup, simulated for high severity (6 washers). Low severity (2 washers) and medium severity (4 washers) configurations are simulated in the same setup, with the only variable being the number of washers used.	14
2.8	Mechanical Looseness: Fourth setup, showing the mechanical looseness for high severity (3 complete turns). In the same setup, mechanical looseness for low (1 complete turn) and medium (2 complete turns) is also simulated.	15
2.9	Bearing fault: third setup, where the healthy ball bearing is replaced with a faulty ball bearing (blue in color, in the middle).	16
2.10	Schematic illustration of constructing a triaxial waveform sample at a given timestamp $t = t_1$ by stacking the X, Y, and Z axis waveforms.	18
6.1	Confusion matrix for the 2D CNN fault classifier on the held-out test set of ten recordings. Diagonal cells indicate correct classifications; the two off-diagonal entries in the Unbalance row correspond to recordings misclassified as Normal. Overall accuracy: 80% (8 of 10 correct).	45

6.2	Confusion matrix from the 1D CNN with LSTM proof-of-concept evaluation. Sample counts are at the window level rather than at the recording level, and an additional Bearing fault class is included that was retained from an earlier iteration of the project’s class taxonomy. Overall window-level accuracy: approximately 90.6%. . . .	47
7.1	End-to-end data flow in the deployed VibeDiag system. An uploaded recording is preprocessed once and dispatched in parallel to all three models; per-window predictions are aggregated through majority voting at both the fault and severity stages before being rendered into the dashboard.	53
7.2	The VibeDiag dashboard with a fault recording uploaded. The five sections, status verdict banner, four metric cards, plain-English summary, diagnostic spectrum, and engineering analysis, are arranged vertically and progressively expose deeper diagnostic detail as the engineer scrolls.	56
7.3	Diagnostic spectrum view from the dashboard. The averaged FFT magnitude spectrum is annotated with vertical markers at $1\times$, $2\times$, and $3\times$ the rotational frequency. The four KPI cards below quantify the integrated energy in each harmonic band and the broadband region above the third harmonic.	57
7.4	Engineering Analysis section of the dashboard. The Bode-style log-log spectrum (left) reveals both narrow harmonic content and broadband energy on a single plot, while the shaft-orbit plot (right) shows the trajectory of the rotor in the plane perpendicular to its axis of rotation. Both representations support a richer qualitative interpretation than the linear-axis spectrum alone.	58

List of Tables

2.1	Illustrative example of the structure of the data collected in the lab.	18
6.1	Fault classification performance on Setup 4 test set.	42
6.2	Per-fault severity classification performance on Setup 4 test set (Macro F1).	43

Chapter 1

Introduction

1.1 Background and Motivation

Rotating machinery sits at the heart of modern industrial operations, powering processes across manufacturing plants, transportation networks, energy generation facilities, and automated production lines. Equipment such as electric motors, centrifugal pumps, reciprocating compressors, steam and gas turbines, and geared transmissions runs for extended periods under heavy mechanical loading. Unexpected failures of these machines lead to interrupted production schedules, expensive emergency repairs, and, in the most serious cases, hazardous conditions for nearby personnel. Sustaining the reliability of such equipment is therefore a primary concern across a wide range of engineering disciplines.

The financial consequences of sudden machinery failure are considerable. Industry analyses estimate that manufacturers collectively absorb tens of billions of dollars in costs each year due to unplanned outages, with broader figures suggesting the global burden on large industrial enterprises exceeds millions of dollars annually when lost output, idle workforce hours, and recovery activities are taken into account. In particularly capital-intensive settings such as large-scale automotive assembly, a single hour of lost production can amount to more than a million. Numbers of this scale make a compelling case for shifting away from purely reactive practices and toward maintenance strategies that anticipate failure.

The two long-established maintenance paradigms, reactive and preventive, both fall short of this goal. A reactive approach responds to failures only after they have already disrupted operation, by which point secondary damage to adjacent components has often occurred. A preventive approach replaces or services parts on a fixed schedule, which improves on reactive practice but ignores the actual mechanical state of each machine and frequently results in either unnecessary servicing or undetected wear progressing between scheduled visits. Neither strategy provides visibility into the early stages of fault development, and neither offers any indication of how rapidly an emerging fault

may progress toward failure.

Predictive maintenance addresses these shortcomings by detecting faults as they begin to develop and by quantifying how advanced each fault has become. The underlying idea can be understood through a familiar parallel from clinical medicine. A physician auscultating a patient with a stethoscope is searching for faint irregularities that signal a developing condition before symptoms become severe, and the same physician must also assess how far the condition has progressed in order to decide on a course of treatment. In a similar way, condition monitoring of rotating machinery seeks both to flag the presence of a developing fault and to characterize its severity, because the two findings together determine the appropriate response. A fault detected at an early, low-severity stage can typically be monitored and addressed during a planned maintenance window, whereas a fault that has reached high severity may demand intervention before the next scheduled stop. Small departures from a machine’s normal vibration behavior frequently appear well in advance of any visible failure, and tracking the magnitude of those departures over time enables maintenance teams to act with the appropriate level of urgency.

Of the various sensing modalities available for condition monitoring, vibration measurement has emerged as one of the most informative and widely deployed. The vibration signature of a rotating machine carries a direct record of its internal mechanical interactions, and specific spectral patterns in this signature correspond to well-understood fault modes such as rotor unbalance, shaft misalignment, bearing degradation, and structural looseness. The amplitude and spread of these spectral patterns additionally reflect the severity of the underlying fault: mild conditions produce small, localized spectral changes, while advanced conditions generate pronounced harmonics, broadband energy increases, or both. Following the clinical analogy once more, a physician will frequently auscultate at multiple points on a patient’s body in order to triangulate a finding; similarly, modern vibration measurement uses triaxial accelerometers to capture motion along three orthogonal axes simultaneously, producing a far richer picture of machine state than any single-axis measurement could provide.

Yet despite their diagnostic richness, vibration signals are not straightforward to interpret. They are typically non-stationary, contain energy across a wide range of temporal scales, and shift noticeably with changes in rotational speed, applied load, and ambient conditions. Severity estimation introduces an additional layer of difficulty, because the same fault type can present at multiple intensity levels with spectral distinctions between adjacent levels that are often subtle relative to the differences between fault types. Conventional vibration-based diagnostic methods address these challenges through carefully engineered features, frequency-domain transformations, and the application of expert judgment. Such approaches perform well in the controlled settings for which they were designed, but they do not always transfer reliably to new machines, new operating regimes, or new fault behaviors. Severity thresholds calibrated for one rig or one machine class frequently fail to generalize, requiring substantial recalibration before deployment elsewhere.

Recent developments in deep learning and time-series classification suggest a complementary

path forward, in which discriminative representations for both fault type and fault severity are learned directly from raw vibration data rather than being prescribed by an analyst. At the same time, the maturation of wireless triaxial vibration sensors and the falling cost of high-resolution data acquisition systems have made it considerably easier to assemble large, labeled datasets covering a wide range of conditions. The present project benefited from high-resolution wireless triaxial sensors supplied by Machinery Monitoring Systems, LLC, which made it possible to collect data efficiently under realistic experimental conditions and across multiple severity levels for each applicable fault type. Taken together, these developments provide both the data and the algorithmic foundations needed to investigate deep-learning-based methods for robust vibration-based fault classification, severity estimation, and predictive maintenance of rotating machinery.

1.2 Problem Statement

While vibration signals offer a rich window into mechanical health, the task of accurately recognizing faults in rotating machinery and assessing how severe each fault has become remains a difficult one. The data show substantial variation as a function of rotational speed, mechanical loading, sensor placement, and the underlying severity of any developing fault. The resulting signals are characteristically non-stationary and reflect intertwined contributions from both time-domain dynamics and frequency-domain structure, which makes individual fault signatures difficult to isolate using conventional analytical techniques alone. Severity prediction compounds the problem further, since the same fault can progress through low, medium, and high stages with spectral distinctions between adjacent levels that are often more subtle than the distinctions between different fault types.

Established diagnostic practice depends on hand-crafted features and expert-led frequency analysis to make sense of vibration patterns. In this paradigm, severity is generally inferred by comparing computed feature values against pre-calibrated threshold bands that have been tuned for a particular machine. Such methods place high demands on domain expertise and are sensitive to shifts in machine configuration, background noise, and operating point. They consequently struggle when applied to unfamiliar environments, to fault behaviors that were not anticipated during feature design, or to severity calibrations developed for a different class of asset.

Deep-learning-based methods address part of this gap by extracting discriminative features directly from raw time-series measurements, reducing the dependence on manual feature design. However, most existing approaches treat fault classification as the sole task and do not produce a parallel severity estimate, leaving practitioners without a key input for deciding how urgently to act on a detected fault. Even when severity is addressed, the typical model operates at a single temporal resolution or commits primarily to either the time domain or the frequency domain. Real mechanical faults, in contrast, exhibit patterns that span several temporal scales simultaneously, and a model that captures only one of those scales risks missing diagnostic content critical to separating

one severity level from another within the same fault type.

These observations point to the need for a single framework capable of extracting discriminative information from high-resolution triaxial vibration recordings, incorporating time-domain and frequency-domain content at multiple temporal resolutions, and producing both a fault classification and a severity estimate in one inference pass. The problem addressed in this project is precisely the design and evaluation of such a deep-learning-based framework for jointly classifying common rotating machinery faults and estimating their severity under controlled experimental conditions.

1.3 Objectives

The central aim of this project is to design and evaluate a deep learning based framework that performs accurate and robust fault classification together with severity estimation for rotating machinery using vibration measurements. In line with the challenges set out in the problem statement, the work emphasizes learning discriminative representations directly from raw sensor signals while reducing dependence on hand-crafted signal-processing heuristics for both the type and the severity of each fault. The specific objectives pursued in this project are as follows:

- To collect and preprocess high-resolution triaxial vibration recordings spanning multiple operating conditions of a rotating machine, with each fault condition further captured at multiple severity levels to support joint fault and severity modelling.
- To benchmark established time-series classification approaches, including Random Forest baselines, one-dimensional Convolutional Neural Networks, two-dimensional CNNs operating on spectrogram inputs, hybrid 1D CNN with LSTM models, ResNet-based architectures, MINIROCKET, and Multi-Scale CNNs, on the tasks of vibration-based fault classification and per-fault severity classification.
- To design a two-stage classification architecture in which a multi-label fault classifier produces independent per-class probabilities through a sigmoid output, and conditional per-fault severity classifiers produce softmax distributions over Low, Medium, and High severity for each detected fault, thereby decoupling the two prediction problems while preserving a single inference pipeline.
- To jointly leverage time-domain waveform information and frequency-domain representations within the architecture, allowing the model to draw on complementary views of the same signal during both fault identification and severity assessment.
- To incorporate multi-scale temporal processing through parallel convolutional kernels of varying receptive fields, enabling the model to capture both narrow harmonic peaks characteristic of certain fault types and broader spectral envelopes associated with others, as well as the graded changes in these patterns that correspond to different severity levels.

- To rigorously assess model performance using appropriate classification metrics across both stages, and to examine robustness across fault types, severity levels, and operating speeds.

Taken together, these objectives are intended to demonstrate the effectiveness of multi-scale, dual-domain feature learning combined with a two-stage classification design for joint fault diagnosis and severity estimation in rotating machinery.

1.4 Scope of the Project

This project addresses vibration-based fault classification and severity estimation for a single rotating machinery configuration operating under controlled experimental conditions. The scope is delineated below in order to make clear what the proposed approach is intended to demonstrate and what it explicitly does not attempt to address.

The scope of the project includes:

- The analysis of triaxial vibration recordings captured from wireless sensors mounted on a custom rotating machinery test rig.
- The classification of five machine conditions, comprising normal operation, bearing fault, unbalance, misalignment, and mechanical looseness, together with the estimation of severity at three discrete levels, Low, Medium, and High, for each fault condition for which severity is applicable.
- Operation across three rotational speeds (2,000, 2,500, and 3,000 RPM), with the dataset stratified jointly across fault type, severity level, and rotational speed during training and evaluation.
- The development, training, and offline evaluation of deep learning and classical time-series models within a unified experimental protocol.
- A systematic comparison of the proposed framework against selected baseline and state-of-the-art classifiers under matched data and evaluation conditions.

The project does not undertake remaining-useful-life prediction or full prognostic modelling of fault progression over time. Real-time field deployment on production machinery, embedded inference on edge hardware, and adaptation to other machine classes or substantially different operating regimes are likewise outside the immediate scope of this work, although the system is designed and documented in a way that supports such future extensions. The analysis is further restricted to vibration measurements and does not draw on other sensing modalities such as acoustic emissions, infrared thermography, lubricant analysis, or motor-current signatures.

By making these boundaries explicit, the project concentrates on establishing the feasibility and effectiveness of deep learning based vibration analysis for combined fault classification and severity estimation, without making claims that extend beyond the experimental setup and the data on which the system has been evaluated.

Chapter 2

Experimental Setup

This chapter presents the experimental framework used for vibration data collection and fault simulation in the rotating machinery system. It describes the four experimental setups developed in the laboratory environment, the data acquisition procedure, and the preprocessing techniques applied to the collected vibration signals prior to analysis. In addition, this chapter explains how different fault classes, including bearing faults, misalignment, mechanical looseness, and unbalance conditions, were intentionally simulated under controlled operating conditions.

Multiple severity levels were also introduced for each fault category to replicate varying degrees of machine degradation and to generate a comprehensive dataset for fault classification and severity prediction tasks. The overall objective of this experimental design was to ensure reliable, repeatable, and realistic vibration data generation for subsequent machine learning analysis.

2.1 Rotating Machinery Test Rig

The experiments were conducted using a laboratory-scale rotating machinery test rig designed to simulate common operating conditions observed in industrial rotating systems. The setup consisted of a motor-driven shaft assembly supported by bearing housing and connected to mechanical components that allowed controlled introduction of different fault conditions while maintaining stable operating behavior.

A wireless triaxial vibration sensor provided by Machinery Monitoring Systems, LLC was mounted on the bearing housing away from the motor to capture vibration responses during machine operation. The sensor recorded acceleration signals along three orthogonal axes, enabling comprehensive monitoring of machine dynamics under both normal and faulty conditions.

The following sections describe the four experimental configurations used to simulate different fault classes and severity levels in the laboratory environment.

2.2 Data Acquisition

Vibration data were collected using a wireless triaxial accelerometer during steady-state operation of the rotating machinery system. The wireless setup allowed flexible sensor placement and efficient data collection without additional cabling constraints.

Acceleration signals were recorded along three orthogonal axes to capture machine vibrations from multiple directions under both normal and faulty operating conditions. Data collection was performed under consistent operating settings to ensure reliable comparison across different fault classes and severity levels.

All vibration signals were stored digitally and organized based on operating condition, fault category, and severity level to support efficient preprocessing, labeling, and subsequent machine learning analysis.

2.3 Four Different Setups

Four different experimental setups were evaluated during this study to identify the most suitable configuration for vibration-based fault classification. The main objective of testing multiple setups was to determine which arrangement could generate stable and distinguishable vibration patterns for all simulated fault conditions.

In the first phase of the project, vibration data were collected using different setup configurations, and separate fault classification models were developed for each setup. The performance of each configuration was evaluated based on its ability to clearly differentiate between normal operation, unbalance, mechanical looseness, misalignment, and bearing fault conditions.

The first three setups showed overlapping vibration characteristics between certain fault classes, which reduced the classification performance of the models. In contrast, the fourth setup produced more distinct vibration responses and allowed clearer separation of all fault categories.

Based on these observations, the fourth setup was selected as the final experimental configuration. All subsequent severity analysis and severity-based data collection were therefore performed using only the fourth setup.

2.3.1 First Setup

Rotor is closer to motor and accelerometer is mounted on the middle bearing.



Figure 2.1. First Setup: rotor on the left of sensor which is mounted on ball bearing.

2.3.2 Second Setup

Rotor is between ball bearing and sleeve bearing, and accelerometer is mounted on right end bearing.

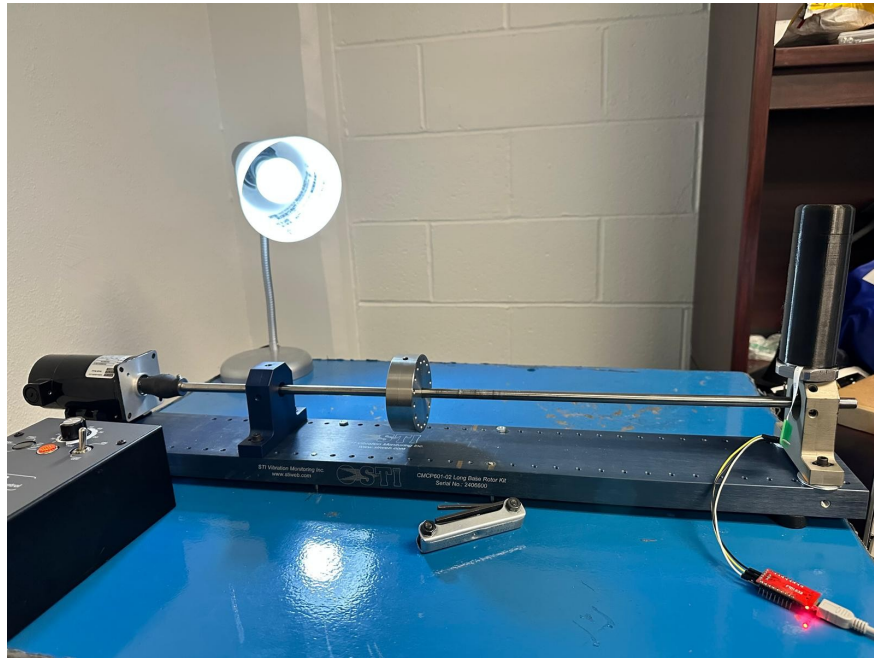


Figure 2.2. Second Setup: rotor on the right of ball bearing and sensor at the end mounted on sleeve bearing.

2.3.3 Third Setup

Rotor is in between ball bearing and sleeve bearing, equally placed, and accelerometer is mounted on right end bearing.



Figure 2.3. Third Setup: rotor on the left of ball bearing and every part is equally distanced.

2.3.4 Fourth Setup (Working Setup)

Rotor between two sleeve bearings and sensor mounted on sleeve bearing at the right end, where every component is equally placed.

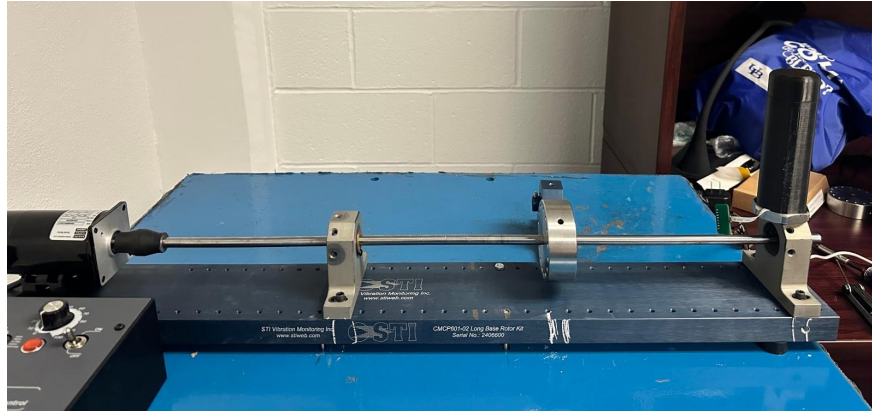


Figure 2.4. Fourth Setup: rotor on the right of sleeve bearing and every component (motor, sleeve bearing, rotor, and sensor at the end) is equally distanced.

2.4 Operating Conditions: Fault Simulation and Severity Simulation

Multiple operating conditions were intentionally simulated on the rotating machinery test rig to generate vibration data for fault classification and severity prediction. Each fault condition was introduced independently while maintaining consistent sensor placement and data acquisition settings throughout the experiments.

Data were collected at three rotational speeds: 2000, 2500, and 3000 RPM. These speeds were selected to provide stable machine operation across a broad operating range. Higher speeds were excluded due to excessive vibration and instability of the experimental setup. Additional experiments were also performed at rotational speeds between 3000 and 5000 RPM. During testing, operating conditions around 3800–4000 RPM were found to be particularly difficult to handle because of excessive vibration, increased machine noise, and noticeable heat generation, indicating unstable behavior of the test rig. Although operation near 5000 RPM appeared relatively more stable than around 4000 RPM, significant vibration and noise were still present. Therefore, higher-speed conditions were excluded, and the study proceeded using the selected stable operating speeds.

For severity analysis, three operating conditions (low, medium, and high) were simulated for each selected rotational speed. For example, in the unbalanced fault condition, vibration data were collected for low severity at all three operating speeds, followed by medium severity at the same three speeds, and similarly for high severity. This resulted in a total of nine operating combinations

for each fault category, including unbalance, mechanical looseness, and shaft misalignment. Normal operating conditions were also recorded across the three selected speeds for baseline comparison.

Bearing fault severity analysis was not included in this part of the study, as the experiments proceeded with the fourth setup configuration and additional sleeve bearings were not available to physically simulate multiple bearing fault severity conditions in the laboratory environment.

2.4.1 Normal Operating Condition



Figure 2.5. Normal Condition: Fourth setup in normal condition where data was collected for 2000, 2500, and 3000 RPM.

The normal operating condition represents the healthy baseline state of the rotating machinery system. In this setup, the machine was operated without introducing any intentional faults, while all mechanical components remained properly aligned and securely fastened. Vibration data collected under this condition represents the behavior of a stable and normally functioning system and were used as the reference for comparison with all simulated fault conditions. For severity analysis, normal operating data were collected at the three selected rotational speeds: 2000, 2500, and 3000 RPM.

2.4.2 Unbalance

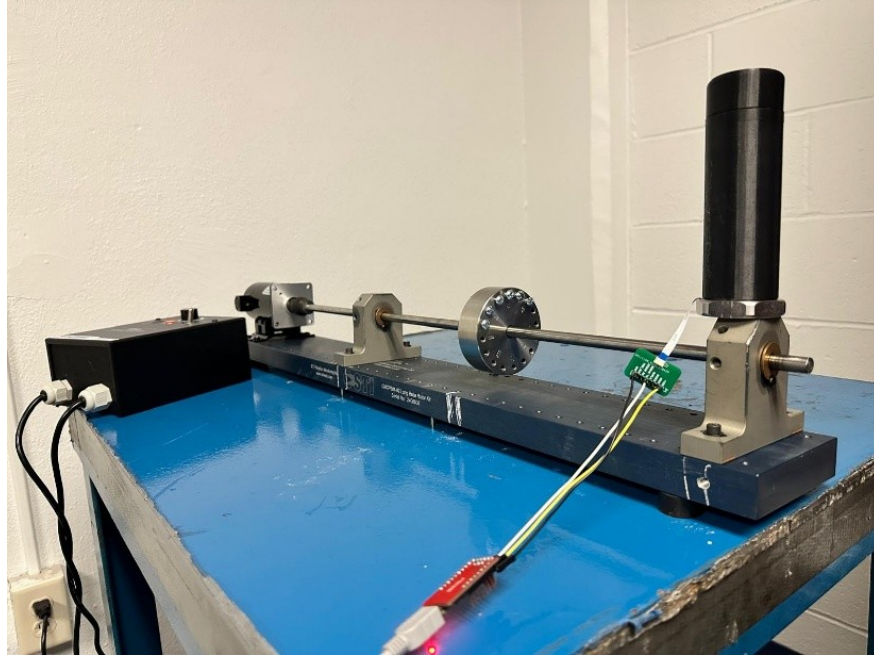


Figure 2.6. Unbalanced condition: shows the unbalanced condition for high severity (6 screws) simulated in the lab. Low (2 screws) and medium (4 screws) configurations are also used to collect data for all severity levels, keeping everything else in the setup the same except for the number of screws.

The unbalanced fault condition was simulated by introducing asymmetric mass distribution on the rotor. This was achieved by attaching screws (weights) at specific locations on the rotor, creating uneven centrifugal forces during machine rotation. As a result, periodic vibration components synchronized with the rotational speed were generated.

Different unbalance conditions were created using two-screw, four-screw, and six-screw configurations. For severity analysis, the two-screw configuration was considered as low severity, the four-screw configuration as medium severity, and the six-screw configuration as high severity. Vibration data for each severity level were collected at the three selected operating speeds: 2000, 2500, and 3000 RPM, resulting in a total of nine operating combinations for the unbalanced condition. Each operating condition was recorded for approximately 15 minutes, generating around 3,000 to 4,000 rows of vibration data stored in JSON format.

2.4.3 Mechanical Misalignment



Figure 2.7. Mechanical misalignment: fourth setup, simulated for high severity (6 washers). Low severity (2 washers) and medium severity (4 washers) configurations are simulated in the same setup, with the only variable being the number of washers used.

Misalignment was simulated by introducing a vertical offset beneath the bearing housing located near the sensor position. This was achieved by placing washers (thin metal plates with a center hole) beneath the bearing support, causing the shaft to deviate slightly from its normal horizontal alignment, as shown in Figure 2.7. The introduced offset altered the load distribution across the shaft and bearings, resulting in increased vibration levels and additional harmonic components.

For severity analysis, three misalignment conditions were considered based on the number of washers used. Two washers represented low severity, four washers represented medium severity, and six washers represented high severity misalignment. Vibration data for each severity level were collected at the selected operating speeds of 2000, 2500, and 3000 RPM, resulting in nine operating combinations for the misalignment fault condition, recorded for 15 minutes each.

2.4.4 Mechanical Looseness Condition



Figure 2.8. Mechanical Looseness: Fourth setup, showing the mechanical looseness for high severity (3 complete turns). In the same setup, mechanical looseness for low (1 complete turn) and medium (2 complete turns) is also simulated.

Mechanical looseness was simulated by gradually loosening the mounting screws of the bearing housing located beneath the sensor, as shown in Figure 2.8. Loosening the assembly introduced additional structural movement and intermittent mechanical contact, resulting in irregular and broadband vibration behavior during machine operation. Compared to unbalanced and misalignment faults, the vibration patterns produced by mechanical looseness were less periodic and appeared more random in nature, making the condition relatively more difficult to classify.

For severity analysis, different levels of looseness were introduced by rotating the mounting screws by a fixed number of complete turns. Low severity was created by loosening the screws by one complete rotation (360°), medium severity by two complete rotations, and high severity by three complete rotations. Vibration data for each severity level were collected at the selected operating speeds of 2000, 2500, and 3000 RPM, resulting in nine operating combinations for the

mechanical looseness condition. All experiments were conducted using the same sensor placement and acquisition settings to maintain consistency across the dataset.

2.4.5 Bearing Fault Condition



Figure 2.9. Bearing fault: third setup, where the healthy ball bearing is replaced with a faulty ball bearing (blue in color, in the middle).

To simulate the bearing fault condition, a defective ball bearing (blue in color) was installed beneath the vibration sensor location, as illustrated in Figure 2.9. The defect consisted of a crack in the inner race of the bearing, which generated repeated mechanical impacts during shaft rotation. These impacts produced impulsive vibration components that propagated through the shaft and bearing housing, making the condition suitable for vibration-based fault diagnosis.

Bearing fault experiments were considered only in the first three experimental setups. In these setups, the system used ball bearings, allowing the healthy bearing to be directly replaced with the defective bearing for fault simulation (one extra faulty ball bearing was already available in the laboratory). However, the fourth setup used sleeve bearings, and additional faulty sleeve bearings were not available in the laboratory environment. Therefore, bearing fault simulation and severity analysis were not included in the fourth setup.

2.5 Collected Raw Data Format and Structure

The wireless triaxial vibration sensor used in this study measured acceleration signals along the X, Y, and Z directions during machine operation. The sensor continuously transmitted raw vibration signals through a serial communication interface, where the data were received as waveform frames.

Each waveform contained 1024 acceleration data points for a single axis. The sensor operated at a sampling rate of 50 kHz, which allowed the system to capture both low-frequency machine vibrations and high-frequency fault-related signals. This was especially useful for detecting impulsive vibration behavior generated by faults such as bearing defects or mechanical looseness.

A custom Python-based acquisition script was developed to collect and organize the incoming vibration data in real time. Since the sensor transmitted data separately for the X, Y, and Z axes, the script was designed to save data only when a complete set of all three axes was received correctly. Any incomplete or incorrectly ordered measurements were automatically discarded to maintain consistency in the dataset.

Before storing the signals, a simple preprocessing step was applied by subtracting the mean value from each waveform to remove constant sensor offsets while preserving the actual vibration behavior of the machine.

The processed vibration signals were stored in CSV files, with separate files created for each operating condition and fault category. Each row in the dataset contained a timestamp, the corresponding axis label (X, Y, or Z), and 1024 time-domain acceleration values. Measurements recorded at the same instant shared a common timestamp, allowing all three axes to be grouped together during later stages of preprocessing and machine learning analysis. The data files are arranged based on different fault types to provide efficient data ingestion and labeling. Each CSV file follows the format:

- A timestamp indicates when the waveform group was recorded in the machine.
- An axis identifier (X, Y, or Z).
- 1024 numerical values representing the time-domain vibration waveform.

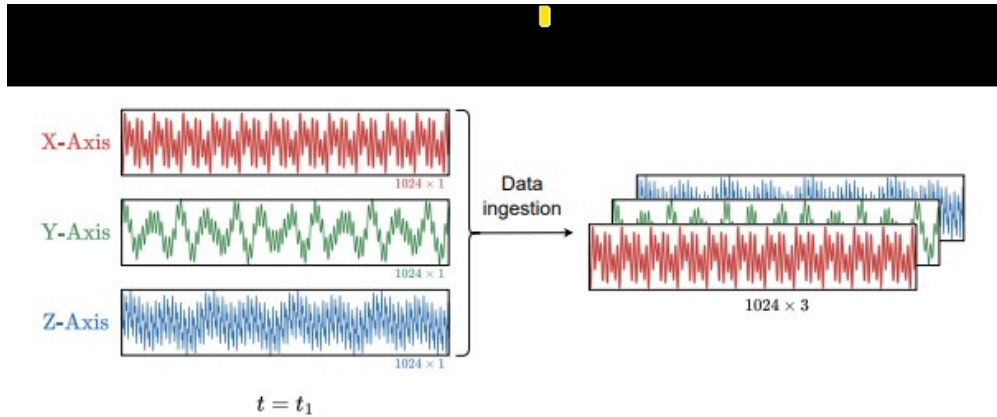
Table 2.1. Illustrative example of the structure of the data collected in the lab.

Timestamp	Axis	V_1	V_2	V_3	V_4	$\dots$	V_{1024}
2026-03-15 10:24:31.421	X	0.0124	-0.0083	0.0156	-0.0098	$\dots$	0.0117
2026-03-15 10:24:31.421	Y	-0.0091	0.0142	-0.0067	0.0085	$\dots$	-0.0103
2026-03-15 10:24:31.421	Z	0.0068	-0.0119	0.0094	-0.0125	$\dots$	0.0089
2026-03-15 10:24:31.442	X	0.0135	-0.0072	0.0148	-0.0089	$\dots$	0.0121
2026-03-15 10:24:31.442	Y	-0.0103	0.0156	-0.0079	0.0091	$\dots$	-0.0115
2026-03-15 10:24:31.442	Z	0.0074	-0.0128	0.0102	-0.0117	$\dots$	0.0093

In Table 2.1, example entries from the collected triaxial vibration dataset are shown. Each waveform contains 1024 time-domain acceleration samples; however, only the first few sample values are displayed here for illustration purposes.

2.6 Data Ingestion and Dataset Construction

After the vibration signals were collected and stored in CSV format, a separate data ingestion step was performed to organize the raw recordings into a structured format suitable for machine learning analysis. This stage acted as a bridge between the experimentally collected vibration signals and the final feature representations used for model development.

**Figure 2.10.** Schematic illustration of constructing a triaxial waveform sample at a given timestamp $t = t_1$ by stacking the X, Y, and Z axis waveforms.

Each CSV file corresponding to a specific operating condition was loaded using Python-based data processing libraries. Since the sensor transmitted data separately for the X, Y, and Z axes,

waveform rows sharing the same timestamp were grouped together to reconstruct a complete triaxial vibration measurement. In this way, the three axis signals were combined into a single waveform sample.

For every timestamp, the X-, Y-, and Z-axis waveforms were stacked together to create a triaxial vibration array of size 1024×3 , where 1024 represents the number of time-domain samples and 3 corresponds to the three sensor axes. This structure preserved both the temporal vibration information and the directional behavior of the machine vibrations across all axes. Figure 2.10 illustrates the construction of a triaxial waveform sample from the three sensor axes.

After constructing the triaxial waveforms for each operating condition, all samples were combined into a single dataset represented as $\mathbf{X}_{\text{waveforms}}$. The final dataset contained 84,119 waveform samples with an overall shape of:

$$\mathbf{X}_{\text{waveforms}} \in \mathbb{R}^{84,119 \times 1024 \times 3}$$

Each waveform sample was then assigned a label corresponding to its operating condition, including normal operation, bearing fault, unbalance, misalignment, and mechanical looseness. Labels were encoded using one-hot vectors to support supervised classification. The combined label matrix, denoted as $\mathbf{Y}$, had a final shape of:

$$\mathbf{Y} \in \{0, 1\}^{84,119 \times 5}$$

At this stage, the dataset consisted of raw triaxial vibration waveforms and their corresponding operating-condition labels. Additional preprocessing and feature extraction steps, including FFT-based frequency-domain transformation and feature standardization, were performed later before model training and are discussed in the following chapter.

Chapter 3

Dataset Preparation and Feature Engineering

This chapter describes how the raw vibration recordings were prepared for model training and how the feature representations consumed by the classifiers were derived. The objective was to convert one-dimensional triaxial accelerometer signals into structured representations suitable for both deep learning and classical time-series classification, while preserving a reproducible experimental protocol that supports both fault classification and per-fault severity estimation.

3.1 Dataset Overview and Notation

The processed dataset comprises two complementary feature views per recording: a time-domain representation derived from the raw aligned waveforms, and a frequency-domain representation derived from those same waveforms, together with associated fault-type and severity labels.

The time-domain dataset is denoted:

$$\mathbf{X}_{\text{waveform}} \in \mathbb{R}^{N \times 1024 \times 3}$$

where $N = 21,758$ is the total number of triaxial samples retained after triplet alignment and quality filtering, 1024 is the number of time-steps per windowed sample, and 3 is the number of physical channels corresponding to the X, Y, and Z accelerometer axes.

Each waveform sample is associated with a structured label pair $(\mathbf{y}_{\text{fault}}, \mathbf{y}_{\text{severity}})$. The fault label $\mathbf{y}_{\text{fault}}$ is a multi-label binary vector of length $K = 4$ corresponding to Bearing Fault, Mechanical Looseness, Misalignment, and Unbalance, with the Normal condition represented as an all-zero vector. The severity label $\mathbf{y}_{\text{severity}}$ is a per-sample categorical label drawn from {Low, Medium, High}, defined only for samples in which a fault is present and conditional on the active fault class.

The frequency-domain representation is denoted:

$$\mathbf{X}_{\text{FFT}} \in \mathbb{R}^{N \times 513 \times 3}$$

where 513 is the number of frequency bins obtained from a real-input Fast Fourier Transform applied per axis. The construction of $\mathbf{X}_{\text{FFT}}$ is described in the next section.

3.2 Frequency-Domain Feature Generation

Vibration faults in rotating machinery produce distinctive spectral signatures: unbalance manifests at the rotational frequency, misalignment at its second harmonic, mechanical looseness across broader spectral bands, and bearing damage as broadband high-frequency energy with characteristic ball-pass frequencies. Surfacing this frequency-domain structure explicitly is easier to learn from than expecting the model to discover it from the raw time-domain signal alone.

For each sample in $\mathbf{X}_{\text{waveform}}$, an FFT was computed independently along each axis. A Hann window was applied prior to the transform to reduce spectral leakage at the boundaries of the windowed segment. A real-input FFT was then applied per axis, producing a 513-bin one-sided magnitude spectrum. Only the magnitude component was retained; the phase component was discarded, consistent with standard practice in vibration analysis where the relative distribution of energy across frequency bands carries the principal diagnostic information.

To compress dynamic range and improve the visibility of low-amplitude harmonics relative to dominant peaks, the magnitudes were passed through a logarithmic transformation, $\log(1 + |X[k]|)$, before being used as features. The FFT and the subsequent log transformation are both deterministic and parameter-free, so computing them prior to the train-test split introduces no risk of information leakage.

3.3 Train-Validation-Test Split Protocol

The dataset was partitioned into training, validation, and test sets following a 70/15/15 split. The partition was performed once with a fixed random seed and applied consistently across all model evaluations in this work, so that every classifier was assessed on the same held-out test partition. The same split indices were applied identically to $\mathbf{X}_{\text{waveform}}$, $\mathbf{X}_{\text{FFT}}$, the fault labels, and the severity labels.

A critical consideration in constructing the split was the joint imbalance of the dataset across three labeling dimensions: fault type, severity level, and operating speed. Some combinations of these three dimensions are several times more populous than others. A random split that ignored this imbalance risked producing a test set whose composition drifted from the training distribution, particularly under-representing rare combinations and producing unstable per-class performance

estimates.

To address this, the partition used **joint stratification on the (fault, severity, speed) tuple**. Stratification on this triple ensures that every combination is represented in the training, validation, and test partitions in the same proportion as in the full dataset, producing stable per-combination evaluation and reducing the variance of reported metrics. The same split key was used for both the multi-label fault classifier and the per-fault severity classifiers, ensuring consistent train/test boundaries across the two stages of the cascaded architecture.

The training partition was used for model fitting. The validation partition was used during deep model training to monitor generalization and to drive learning-rate scheduling and early stopping. The test partition was held out from all training and hyperparameter decisions, and was used only to compute the final reported performance metrics.

The resulting partition sizes were approximately:

- Training: 15,230 samples
- Validation: 3,264 samples
- Test: 3,264 samples

3.4 Feature Scaling (Standardization)

Following the split, feature scaling was applied to both $\mathbf{X}_{\text{waveform}}$ and $\mathbf{X}_{\text{FFT}}$ to improve numerical stability during training and to balance the contribution of each channel.

Standardization parameters, per-axis means and standard deviations, were computed **exclusively from the training partition**. The fitted parameters were then applied to the validation and test partitions. This ordering is essential: fitting the scaler on the full dataset before splitting would leak distributional information from the test set into the training procedure, inflating reported performance. Fitting only on training data preserves the integrity of the held-out evaluation.

Scaling was performed on a per-axis basis. For each of the three accelerometer channels, an independent mean and standard deviation were estimated from the training partition and used to z-score the corresponding channel across all partitions. Per-axis standardization is appropriate because the three axes can exhibit different signal magnitudes depending on the orientation of the sensor relative to the rotor’s principal modes of vibration.

The fitted scalers were serialized as joblib artifacts and saved alongside the trained model files. At inference time, the same scaler files are loaded and applied to incoming samples before any model is consulted. This guarantees byte-identical preprocessing between training and serving, eliminating training-serving skew as a source of degraded production performance.

At the conclusion of this preprocessing pipeline, the standardized time-domain and frequency-domain datasets are ready for consumption by the learning models described in the subsequent chapters. The 1D CNN, ResNet, Multi-Scale CNN, and 1D CNN + LSTM models operate on the

standardized frequency-domain features $\mathbf{X}_{\text{FFT}}$; MINIROCKET operates directly on the standardized time-domain waveforms $\mathbf{X}_{\text{waveform}}$; the 2D CNN operates on a separately computed spectrogram representation; and Random Forest operates on a set of hand-engineered statistical features derived from both domains. The two-stage architecture proposed in this work uses these same standardized representations for both fault classification and per-fault severity estimation, ensuring consistent feature semantics across the cascade.

Chapter 4

Models and Architectures

This chapter presents the machine learning models and deep learning architectures evaluated in this study for vibration-based fault classification and per-fault severity estimation. All models are designed to map vibration measurements obtained from the experimental setup to two categorical outputs: a multi-label prediction of which fault types are present and, conditional on each detected fault, a categorical prediction of the severity at which that fault is developing.

4.1 Model Input-Output Formulation

The learning task addressed in this chapter is formulated as a two-stage supervised classification problem. Given vibration measurements from a rotating machine operating at a known rotational speed, the objective is to first identify which fault classes are present in the recording, and then, for each detected fault, to estimate its severity level.

Each data sample, indexed by i , is associated with two complementary representations derived from the same physical vibration event:

- a time-domain representation, $\mathbf{X}_{\text{waveform}}[i] \in \mathbb{R}^{1024 \times 3}$, capturing the standardized triaxial vibration waveform across the X, Y, and Z accelerometer axes, and
- a frequency-domain representation, $\mathbf{X}_{\text{FFT}}[i] \in \mathbb{R}^{513 \times 3}$, capturing the log-magnitude FFT spectrum of the same vibration signal.

In addition, each sample is associated with a scalar auxiliary input $s[i] \in \mathbb{R}$ representing the standardized rotational speed at which the recording was captured.

The target output for each sample consists of two components, corresponding to the two stages of the classification cascade:

Stage A — Fault prediction. A multi-label binary vector of length $K = 4$ indicating which faults are present:

$$\hat{\mathbf{y}}_{\text{fault}} \in [0, 1]^4$$

The four fault classes are Bearing Fault, Mechanical Looseness, Misalignment, and Unbalance. The Normal condition is encoded as the all-zero vector. Multiple components of $\hat{\mathbf{y}}_{\text{fault}}$ may be simultaneously above the decision threshold, reflecting the architectural capacity to detect overlapping faults.

Stage B — Per-fault severity prediction. For each fault class c that is predicted positive in Stage A, a categorical distribution over three severity levels:

$$\hat{\mathbf{y}}_{\text{severity},c} \in \Delta^3$$

where the simplex is over {Low, Medium, High}. The predicted severity for fault c corresponds to the class with the highest posterior probability.

At a high level, the models considered in this chapter therefore implement two coupled mappings:

$$(\mathbf{X}_{\text{FFT}}[i], \mathbf{X}_{\text{waveform}}[i], s[i]) \rightarrow \hat{\mathbf{y}}_{\text{fault}} \rightarrow \{\hat{\mathbf{y}}_{\text{severity},c}\}_{c:\hat{y}_{\text{fault},c} > \tau}$$

where τ is the fault detection threshold. Different models in this study consume different subsets of the available inputs: most deep models use $\mathbf{X}_{\text{FFT}}$ and s , MINIROCKET uses $\mathbf{X}_{\text{waveform}}$ directly, and the Random Forest baseline uses hand-engineered features derived from both domains.

4.2 Motivation for Dual-Domain Inputs

The decision to retain both time-domain and frequency-domain representations of every recording is motivated by the physical characteristics of vibration signals in rotating machinery. Mechanical faults manifest through a combination of transient temporal patterns and structured spectral signatures, and no single representation captures both equally well.

Time-domain waveforms preserve impulsive events, amplitude modulations, and short-duration transients of the kind generated by bearing impacts, sporadic contact in loose mechanical assemblies, and other phenomena that are localized in time. These signatures can be attenuated or smeared across multiple frequency bins when viewed purely in the frequency domain, particularly when the underlying event is short relative to the windowing length used in the FFT.

Frequency-domain representations, by contrast, emphasize harmonic structure, the distribution of energy across spectral bands, and the fault-specific periodic components that are closely tied to rotational speed and fault geometry. Unbalance and misalignment in particular produce strong harmonic peaks at integer multiples of the rotational frequency, which are more discriminative in the frequency domain than in the time domain. The frequency-domain view also offers a more compact representation of these periodic phenomena, since a single spectral peak summarizes a pattern that would otherwise be distributed across the entire time-domain waveform.

Severity prediction adds a further motivation for retaining both views. Within a single fault

type, the spectral distinctions between Low, Medium, and High severity levels are often subtle: the location of the dominant peak may remain unchanged while its amplitude grows, the surrounding harmonics may become more prominent, or broadband noise may rise relative to the dominant signal. Capturing these graded changes reliably benefits from access to both representations, since amplitude and temporal density information from the time domain complements the spectral structure information from the frequency domain.

By making both representations available across the model lineup, this study isolates the contribution of each representation to fault and severity classification, and allows architectures that consume only one of the two to be evaluated against architectures that consume both. The choice of which representation a given model uses is described in the architectural sections that follow.

4.3 High-Level Model Overview

All models evaluated in this chapter share a common abstract interface: each acts as a classifier that ingests one or both vibration representations and outputs predictions for fault type and, conditionally, severity. The differences among the models lie in how they internally process those inputs, through learned filters of fixed scale, through learned filters of multiple parallel scales, through random fixed projections followed by a linear classifier, or through hand-engineered features followed by a non-parametric classifier.

The seven architectures evaluated in this study are:

- **Random Forest with hand-engineered time-domain and frequency-domain features** — a non-parametric baseline that tests whether textbook vibration analysis features alone are sufficient for the task.
- **1D Convolutional Neural Network on frequency-domain features** — a deep learning baseline that tests whether a single sliding kernel along the spectral axis can capture fault signatures.
- **1D Residual Network (ResNet) on frequency-domain features** — tests whether convolutional depth combined with residual connections improves over the plain 1D CNN baseline.
- **1D CNN combined with Long Short-Term Memory (LSTM)** — a hybrid architecture that pairs convolutional feature extraction with a recurrent module, testing whether explicit temporal sequence modeling improves classification.
- **2D Convolutional Neural Network on spectrogram representations** — operates on a separately computed time-frequency representation and tests whether learning two-dimensional patterns over time-frequency outperforms one-dimensional models.
- **MINIROCKET with Ridge Classifier** — applies a large bank of fixed random convolutional kernels to the raw time-domain waveform, followed by a linear classifier. Tests whether learned

filters are necessary at all on this dataset.

- **Multi-Scale Convolutional Neural Network on frequency-domain features** — uses parallel convolutional branches with different kernel sizes operating on the same input, testing whether faults manifest at multiple spectral resolutions simultaneously.

Each architecture is evaluated for both Stage A (multi-label fault classification with sigmoid output) and Stage B (per-fault severity classification with softmax output). Stage B is trained and evaluated independently for each fault class, using only the subset of training data in which that fault is present. The cascaded inference procedure, in which Stage B is invoked conditionally on Stage A’s prediction, is described in detail in Chapter 5.

While the individual architectures differ in how they process their inputs internally, this common formulation provides a consistent framework for evaluating them on a single benchmark. The following sections describe each model in detail, beginning with the classical baselines and progressing to the deep learning architectures and finally to the kernel-transform approach.

4.4 Baseline Models

The baseline models evaluated in this study are representative architectures commonly used in time-series classification and vibration-based fault diagnosis. Random Forest provides a non-parametric reference that operates on hand-engineered statistical features and represents the classical analytics approach to vibration monitoring. The 1D CNN provides a deep learning reference that learns features directly from the FFT spectrum and represents the most widely used architecture in the recent vibration classification literature. Each subsequent baseline progressively explores a distinct architectural axis, depth with skip connections (ResNet), convolutional-recurrent hybrids (1D CNN + LSTM), and two-dimensional time-frequency representations (2D CNN).

The selected baselines together establish a strong frame of reference against which the proposed Multi-Scale CNN and the MINIROCKET kernel-transform approach are evaluated. The detailed architecture of each baseline is described in the following subsections.

4.4.1 1D Convolutional Neural Network

The first baseline model evaluated in this study is a one-dimensional convolutional neural network (1D CNN) operating on the frequency-domain representation of the vibration signal. This architecture serves as a strong baseline because convolution along the frequency axis naturally captures the localized harmonic structure that distinguishes one fault type from another, and because the 1D CNN is the most widely used architecture in the time-series vibration classification literature.

The model accepts two inputs derived from a single vibration event:

- a frequency-domain magnitude spectrum input $\mathbf{X}_{\text{FFT}}[i] \in \mathbb{R}^{513 \times 3}$, and

- a scalar auxiliary input $s[i] \in \mathbb{R}$ representing the standardized rotational speed at which the recording was captured.

Frequency-Domain Branch. The main computational branch operates on the FFT magnitude representation and is composed of three convolutional blocks with progressively increasing filter depth. The blocks use 32, 64, and 128 filters respectively, each with a kernel size of 7 samples along the frequency axis. This kernel size was selected to span several adjacent frequency bins simultaneously, capturing local harmonic structures such as a fundamental peak together with its first one or two harmonics in a single receptive field. Each convolutional layer is followed by Batch Normalization to stabilize training and a ReLU activation, with a max-pooling operation of factor 2 applied at the end of each block to progressively reduce frequency resolution and introduce translation invariance along the spectral axis.

A 1D Global Average Pooling layer is applied at the end of the convolutional stack to aggregate the learned feature maps into a fixed-length feature vector of dimension 128. Global average pooling rather than flattening was chosen to reduce parameter count, mitigate overfitting on the modestly sized dataset, and produce a representation that is invariant to the absolute position of features within the spectrum.

Speed Feature Injection. The scalar speed input $s[i]$ is concatenated with the global-pooled feature vector before entering the classification head. This injection allows the network to condition its predictions on rotational speed, which is physically motivated: the same fault type produces spectral peaks at frequencies that scale linearly with rotational speed, and providing the model with explicit speed information enables it to interpret a given spectral pattern in the appropriate rotational context.

Fault Classification Head. The combined feature vector is passed through a fully connected classification head composed of two dense layers with 128 and 64 units respectively, each using ReLU activations and followed by a Dropout layer with rate 0.4 for regularization. The final output layer produces a vector of length $K = 4$ with sigmoid activations, yielding independent per-class probabilities suitable for multi-label fault classification:

$$\hat{\mathbf{y}}_{\text{fault}} \in [0, 1]^4$$

The use of sigmoid rather than softmax is essential to the design: it permits multiple fault classes to be predicted simultaneously, supporting the multi-label formulation in which a sample may carry more than one fault. A sample with all four sigmoid outputs below the classification threshold is assigned to the Normal class.

Severity Classification Heads. A parallel set of severity classifiers, one per fault class, share the same convolutional backbone and pooling layers but maintain independent classification heads. Each severity head produces a softmax distribution over three classes, Low, Medium, and High, for samples in which the corresponding fault has been detected by the upstream fault classifier. The

training procedure and the conditional inference protocol that uses these severity heads only when the fault classifier flags a positive prediction are described in Chapter 5.

4.4.2 2D Convolutional Neural Network on Spectrogram Representations

The next baseline model evaluated in this study is a two-dimensional convolutional neural network (2D CNN) that operates on a time-frequency representation of the vibration signal rather than on the one-dimensional frequency spectrum used by the other deep models. This architecture tests the hypothesis that fault signatures, particularly those that vary in their spectral content over time, are more discriminative when viewed as a two-dimensional image-like representation in which both temporal evolution and spectral structure are visible simultaneously.

Input Representation. For each triaxial vibration recording, a Short-Time Fourier Transform (STFT) is computed independently along each accelerometer axis. The STFT decomposes the signal into overlapping time windows, applies a Hann window to each, and computes the magnitude spectrum of each window. The result is a two-dimensional matrix indexed by time bin and frequency bin, capturing how the spectral content of the signal evolves over the duration of the recording. Stacking the three per-axis spectrograms along a channel dimension produces a three-channel time-frequency representation analogous in structure to an RGB image, which the 2D CNN consumes directly.

Architecture. The model is implemented in PyTorch and consists of a sequence of two-dimensional convolutional blocks followed by a fully connected classification head. Each convolutional block applies a Conv2D layer with kernels of size 3×3 , followed by Batch Normalization, a ReLU activation, and a 2×2 max-pooling operation that halves both the temporal and spectral dimensions. The number of filters increases progressively across the blocks (typically 32, 64, and 128), allowing the network to learn increasingly complex compositions of low-level time-frequency features. A 2D Global Average Pooling layer at the end of the convolutional stack aggregates the learned feature maps into a fixed-length feature vector that is invariant to the absolute position of features within the time-frequency plane.

Fault Classification Head. The pooled feature vector is passed through a fully connected classification head with one hidden dense layer using ReLU activation and Dropout regularization, followed by an output layer that produces the multi-label fault prediction. The fault classifier follows the same multi-label formulation used elsewhere in this study, with independent sigmoid outputs over the four fault classes Bearing, Mechanical Looseness, Misalignment, and Unbalance, and the Normal condition encoded as the all-zero vector. Three additional severity classifiers, one per fault class, share the convolutional backbone but maintain independent dense classification heads, in keeping with the cascaded two-stage design described earlier in this chapter.

Motivation for the Two-Dimensional Approach. The 1D CNN, ResNet, Multi-Scale CNN, and MINIROCKET architectures all consume representations that have collapsed temporal evolution

into a single aggregate view, either by averaging spectra across the recording or by passing the raw waveform through pooled feature extractors. A two-dimensional spectrogram, in contrast, preserves the temporal axis of the signal and allows the model to learn that certain fault signatures evolve over time, that bearing faults produce intermittent broadband impulses rather than steady-state spectral content, or that mechanical looseness can exhibit non-stationary energy distributed across both axes. Whether this richer representation translates into improved classification on the present dataset is an empirical question addressed in Chapter 6.

Training Procedure. The 2D CNN was trained using the Adam optimizer with the same loss functions used for the other deep models in this study: binary cross-entropy with per-class positive weighting for the fault classifier, and sparse categorical cross-entropy with class balancing for each per-fault severity classifier. Early stopping on the validation loss with a patience of eight epochs was used to terminate training automatically at the point of diminishing returns. The trained weights, the fitted preprocessing scalers, and the STFT configuration metadata are serialized as PyTorch state-dictionary files and loaded by the deployed inference engine at runtime.

4.4.3 Multi-Scale Convolutional Neural Network

The second model evaluated in this study is the Multi-Scale Convolutional Neural Network (Multi-Scale CNN), an extension of the 1D CNN baseline designed to address a fundamental limitation of single-kernel-size architectures when applied to vibration spectra.

Motivation and Design Principle. Different fault types in rotating machinery produce spectral signatures at different spectral resolutions. Unbalance, for example, manifests as a narrow peak at exactly $1\times$ the rotational frequency, occupying only a few frequency bins. Misalignment produces sharper peaks at $2\times$ and higher harmonics. Mechanical looseness, by contrast, generates broadband spectral energy distributed across a wider band of frequencies. A 1D CNN with a single fixed kernel size is structurally constrained to view all of these patterns through the same temporal receptive field, which can lead to under-resolution of the narrow features or under-coverage of the broad ones depending on the kernel size selected. This is particularly problematic for severity prediction within a single fault, where the spectral differences between Low, Medium, and High severity levels are often smaller than the differences between fault types and rely on capturing subtle patterns at multiple scales simultaneously.

The Multi-Scale CNN addresses this through parallel convolutional branches with different kernel sizes operating on the same input.

Architecture. The model accepts the same two inputs as the 1D CNN: the FFT magnitude representation $\mathbf{X}_{\text{FFT}}[i] \in \mathbb{R}^{513 \times 3}$ and the standardized scalar speed $s[i] \in \mathbb{R}$. After an initial Batch Normalization layer applied to the input spectrum, the signal is fed in parallel into three convolutional branches, each with a different kernel size:

- Branch A: Conv1D with kernel size 3, 64 filters

- Branch B: Conv1D with kernel size 7, 64 filters
- Branch C: Conv1D with kernel size 15, 64 filters

The three kernel sizes were chosen to span the range of spectral feature widths observed in the dataset. The size-3 branch captures very narrow peaks such as the fundamental harmonic of unbalance. The size-7 branch captures moderate-width features such as misalignment harmonics with their surrounding spectral skirts. The size-15 branch captures broad spectral envelopes characteristic of looseness and bearing wear, where energy is distributed across many adjacent frequency bins.

Each branch follows its initial convolution with Batch Normalization, a ReLU activation, and a max-pooling layer of factor 2. A second convolutional block within each branch then doubles the filter depth to 128, applies the same kernel size as before within that branch, and reduces resolution once more by max-pooling. A 1D Global Average Pooling layer at the end of each branch produces a per-branch feature vector of dimension 128.

Feature Fusion. The three per-branch feature vectors are concatenated into a unified multi-resolution feature representation of dimension 384. The standardized scalar speed input is then concatenated to this vector to provide rotational context, exactly as in the 1D CNN baseline. The resulting 385-dimensional combined feature vector captures complementary information from three distinct spectral resolutions and the operating speed of the machine, all derived from the same FFT input.

Fault and Severity Classification Heads. The combined feature vector is passed through a fully connected classification head with two dense layers of 128 and 64 units respectively, each with ReLU activations and followed by Dropout with rate 0.5. As in the 1D CNN, the fault head produces a vector of length $K = 4$ with sigmoid activations for multi-label fault classification, while parallel severity heads, one per fault class, produce softmax distributions over Low, Medium, and High severity. The conditional inference protocol applies the severity head corresponding to whichever fault class was detected at the threshold by the upstream fault classifier.

Empirical Justification. The parallel-kernel design was empirically validated on multi-fault test cases where two faults were present simultaneously in the same recording. Among the deep models evaluated, only the Multi-Scale CNN consistently detected the secondary fault, while the single-kernel 1D CNN reliably reported only the dominant fault. This observation, together with the improved per-fault severity scores reported in Chapter 6, confirms that the additional architectural complexity of the multi-scale design captures fault-discriminative information that a single-scale architecture cannot.

4.4.4 MINIROCKET with Ridge Classifier

The third model evaluated in this study is MINIROCKET (MINIally RandOm Convolutional KErnel Transform), a recently proposed time-series classification method that takes a fundamentally different approach from the deep learning architectures described above. MINIROCKET does not

learn convolutional filters from data; instead, it applies a large bank of fixed, randomly generated convolutional kernels to the input signal and uses the resulting features as input to a linear classifier. The motivation for evaluating MINIROCKET in this study is twofold: first, it serves as a strong baseline that requires no gradient-based training and runs in a fraction of the time of deep models; second, it directly tests the hypothesis that on this dataset, the patterns required to distinguish fault types and severity levels may be representable in a sufficiently rich random feature space without the need for learned filters at all.

Input Representation. Unlike the 1D CNN and Multi-Scale CNN, MINIROCKET operates directly on the time-domain waveform rather than on the frequency-domain spectrum. The input is therefore $\mathbf{X}_{\text{waveform}}[i] \in \mathbb{R}^{1024 \times 3}$, the standardized triaxial accelerometer signal at native temporal resolution. This is a deliberate choice motivated by the design of MINIROCKET, which computes its own implicit time-frequency representation through its random convolutional basis, internally capturing both impulse-like and periodic structure without requiring a separate FFT preprocessing step. The scalar speed feature is not consumed by MINIROCKET; the random kernels operate uniformly across all samples regardless of the operating speed of the underlying machine.

Random Kernel Transformation. MINIROCKET applies a bank of 10,000 random convolutional kernels to the input waveform. Each kernel has a small fixed length (typically 9 samples) and uses fixed weight values drawn from a small predefined set. The dilations and biases of the kernels are likewise drawn from predetermined distributions designed to span a wide range of temporal scales and offsets. The signal is convolved with each kernel along the time axis, producing a per-kernel response. From each response, a single scalar feature is computed using the Proportion of Positive Values (PPV) statistic, which captures the fraction of the convolved signal that exceeds a per-kernel bias threshold.

The PPV statistic has a key property: it is bounded between 0 and 1, normalizes naturally across signals of varying amplitude, and aggregates information about both the magnitude and temporal density of pattern occurrences. Applied to all 10,000 kernels, the transformation produces a 10,000-dimensional feature vector per input sample. For triaxial signals, the kernel bank is applied per channel, producing a feature matrix that is then flattened.

Ridge Classifier. The 10,000-dimensional MINIROCKET feature vector is passed to a Ridge Classifier with cross-validated regularization (RidgeClassifierCV in scikit-learn, with the regularization parameter α selected from a logarithmic grid over $[10^{-3}, 10^3]$). The Ridge classifier fits a linear model with L2-regularized least-squares loss and operates in closed form without gradient descent. With 10,000 features and 21,758 training samples, the Ridge problem is well-conditioned and the linear classifier converges rapidly.

Multi-Label Adaptation. Because Ridge classification is fundamentally a single-label method, multi-label classification was achieved by wrapping the base Ridge classifier in a one-versus-rest configuration via scikit-learn’s MultiOutputClassifier. This trains an independent binary Ridge classifier for each of the $K = 4$ fault classes, with the per-class output decision-function score

interpreted via a sigmoid as a fault probability:

$$\hat{\mathbf{y}}_{\text{fault}} \in [0, 1]^4$$

This formulation is structurally analogous to the sigmoid output of the deep models, supporting multi-label predictions in which a sample may carry more than one fault.

Severity Prediction. For each fault class, an independent MINIROCKET severity classifier is trained using only samples in which that fault is present. Severity prediction is performed with a standard single-output Ridge classifier (without the one-versus-rest wrapper), producing predictions over the three severity classes Low, Medium, and High. The conditional inference protocol that activates each severity classifier only when its corresponding fault is detected by the fault stage is described in Chapter 5.

Computational Characteristics. MINIROCKET is substantially faster than the deep models. Training the full fault classifier on the training partition completes in approximately five minutes on a single CPU core, compared to thirty to sixty minutes for the deep models. Inference latency per sample is comparable to the deep models, under fifty milliseconds per triplet, because the bottleneck at inference time is the kernel transformation rather than the linear classifier itself.

Chapter 5

Training Setup and Evaluation Metrics

This chapter describes the training procedure used to fit the models presented in Chapter 4, and the evaluation metrics used to assess their performance on the held-out test set. Because the proposed system is structured as a two-stage cascade, a multi-label fault classifier followed by per-fault severity classifiers, the training and evaluation protocol must be specified for each stage independently while ensuring that the two stages share a consistent data partition and a consistent set of preprocessing artifacts. This chapter sets out that protocol in detail.

5.1 Training Configuration

5.1.1 Loss Functions

The two stages of the cascade use different loss functions reflecting their distinct prediction tasks.

Stage A — Fault Classification. The multi-label fault classifier is trained with binary cross-entropy summed across the $K = 4$ fault classes. For a sample with true label vector $\mathbf{y} \in \{0, 1\}^K$ and predicted probabilities $\hat{\mathbf{y}} \in [0, 1]^K$, the loss is:

$$\mathcal{L}_{\text{BCE}}(\mathbf{y}, \hat{\mathbf{y}}) = - \sum_{k=1}^K w_k^+ \cdot y_k \cdot \log(\hat{y}_k) + (1 - y_k) \cdot \log(1 - \hat{y}_k)$$

The per-class positive weight w_k^+ is derived from the training set distribution as the ratio of negative to positive samples for class k . This weighting compensates for the imbalance between samples in which a given fault is present and samples in which it is absent. Without such weighting, the loss would be dominated by the more numerous negative-class examples and would converge to a degenerate solution that predicts the negative class for most samples.

Binary cross-entropy with independent per-class sigmoid outputs is the appropriate loss for the multi-label setting because it treats each fault class as an independent binary decision. This is in contrast to softmax cross-entropy, which assumes mutual exclusivity between classes and would not

permit the simultaneous detection of overlapping faults.

Stage B — Severity Classification. Each per-fault severity classifier is trained with sparse categorical cross-entropy over the three severity levels Low, Medium, and High. For a sample with true severity index $s \in \{0, 1, 2\}$ and predicted distribution $\hat{\mathbf{s}} \in \Delta^3$, the loss is:

$$\mathcal{L}_{\text{CCE}}(s, \hat{\mathbf{s}}) = -\log(\hat{s}_s)$$

Each severity classifier is trained only on the subset of training data in which its corresponding fault is present. The severity model for the Mechanical Looseness class, for example, never sees samples in which the dominant fault is Misalignment or Unbalance. This per-fault training regime ensures that each severity classifier specializes in the spectral distinctions specific to its assigned fault and is not confused by the inter-fault variability that the fault classifier in Stage A is designed to capture.

Within each severity classifier, sklearn-balanced class weights are applied to handle imbalance among Low, Medium, and High samples within the per-fault subset. The weights are computed as the inverse of the per-class frequency in the training subset, scaled so that the sum of weights equals the number of classes.

5.1.2 Optimizer and Learning Rate

All deep learning models in this study are trained with the Adam optimizer with an initial learning rate of 1×10^{-4} . Adam was selected for its adaptive per-parameter learning rates and its robustness to mini-batch noise, both of which are particularly useful given the modest size of the training partition.

A learning-rate scheduling policy is applied through the ReduceLROnPlateau callback. When the validation loss fails to improve for four consecutive epochs, the learning rate is multiplied by 0.5, with a minimum learning rate floor of 1×10^{-6} . This policy allows the model to make rapid progress during the early phase of training when the loss landscape is steep, and then to refine its parameters at progressively smaller step sizes as the optimization approaches a local minimum.

For the MINIROCKET model, no gradient-based optimization is required. The 10,000 random kernels are fixed by construction, and the downstream Ridge classifier is fit in closed form by solving a regularized least-squares system. The regularization parameter α is selected via internal cross-validation over a logarithmic grid spanning $[10^{-3}, 10^3]$.

For the Random Forest baseline, hyperparameters were selected based on standard defaults in the time-series literature: 200 trees, a maximum depth of 20, with no further hyperparameter tuning performed.

5.1.3 Batch Size and Epochs

The deep learning models are trained with a batch size of 32 and a maximum of 30 epochs. The batch size of 32 was chosen as a balance between training stability and memory efficiency on CPU-only hardware. The 30-epoch maximum is rarely reached in practice; convergence typically occurs between epochs 10 and 20, and training terminates at that point via the early-stopping mechanism described below.

5.1.4 Training Callbacks

Two callbacks govern the training of the deep learning models:

- **EarlyStopping** monitors the validation loss with a patience of 8 epochs. When the validation loss fails to improve for 8 consecutive epochs, training halts. The model weights are restored to those from the epoch with the lowest observed validation loss, rather than retaining the weights from the final epoch. This mechanism is essential for preventing overfitting on the training partition while still allowing the optimizer enough flexibility to escape transient validation-loss plateaus.
- **ReduceLROnPlateau** monitors the validation loss with a patience of 4 epochs. Triggered before EarlyStopping, it gives the optimizer a chance to refine its position at a smaller learning rate before the early-stopping criterion concludes that no further progress is possible.

The combination of EarlyStopping and ReduceLROnPlateau produces a training procedure that terminates automatically at the point of diminishing returns, without requiring manual epoch-count tuning per architecture.

5.1.5 Reproducibility

All experiments use fixed random seeds across the libraries involved: `numpy.random.seed(42)`, `tensorflow.random.set_seed(42)`, and `random_state=42` for all scikit-learn estimators. This ensures that the train-validation-test split, the initialization of model weights, the random kernel parameters for MINIROCKET, and the order of mini-batch sampling are all reproducible across runs.

The fitted scalers, the trained model weights, and the configuration metadata are all serialized as joblib or Keras-native artifacts at the conclusion of training, making the deployed system byte-identical with the system on which the reported metrics were computed.

5.2 Evaluation Metrics

Performance is reported separately for each stage of the cascade, using metrics appropriate to the prediction task at each stage.

5.2.1 Accuracy

Accuracy is the proportion of correctly classified samples. For Stage A (multi-label fault classification), accuracy is computed per fault class as the proportion of samples for which the binarized prediction (above or below the 0.5 threshold) matches the ground-truth label. For Stage B (single-label per-fault severity), accuracy is the proportion of severity-level samples for which the argmax of the softmax output equals the ground-truth severity index.

Accuracy is intuitive but can be misleading when class distributions are imbalanced. For a fault class present in only 25% of samples, a degenerate classifier that always predicts the negative class achieves 75% accuracy without producing any useful detections. For this reason, accuracy is reported alongside Macro F1 throughout this study, which is more diagnostic in the presence of imbalance.

5.2.2 Macro F1-Score

The F1-score for a single class is the harmonic mean of precision and recall:

$$F1_k = \frac{2 \cdot P_k \cdot R_k}{P_k + R_k}$$

where P_k is the proportion of predicted positives for class k that are correctly positive, and R_k is the proportion of true positives that are correctly identified.

The Macro F1-score is the arithmetic mean of per-class F1 scores:

$$\text{Macro F1} = \frac{1}{K} \sum_{k=1}^K F1_k$$

Macro F1 is the primary metric used in this study for both stages. By averaging across classes without weighting by class frequency, it penalizes models that perform poorly on minority classes, a critical property for safety-relevant applications such as fault detection, where the rare and severe cases are precisely the ones most consequential to identify correctly.

5.2.3 Confusion Matrix

The confusion matrix $C \in \mathbb{N}^{K \times K}$ for a multi-class problem reports the number of samples with true class i predicted as class j . Diagonal dominance indicates a well-calibrated classifier; off-diagonal mass reveals which classes the model tends to confuse.

For Stage A, multi-label confusion is reported per-class as a 2×2 matrix (true positive, false positive, false negative, true negative). For Stage B, the confusion matrix is the full 3×3 (Low, Medium, High) within each per-fault subset, allowing inspection of whether errors tend to be one-step-off (e.g., predicting Low when the truth is Medium) or two-step-off (e.g., predicting Low

when the truth is High).

The latter distinction is operationally important. A one-step error has limited consequence for maintenance scheduling, since adjacent severity levels imply similar urgency. A two-step error, in which a High-severity fault is reported as Low, represents a more serious failure of the system because it could delay a maintenance action that is genuinely time-critical.

5.2.4 Per-Class Precision, Recall, and F1

In addition to the macro-averaged metrics, per-class precision, recall, and F1 are reported for every class in both stages. This breakdown reveals whether the model performs uniformly across all fault types and severity levels, or whether certain classes are systematically under-served.

Per-class precision is particularly important for severity classification. A severity classifier that has high recall for Low severity but low precision is producing many false alarms, a behavior that, repeated across a fleet of machines, would erode operator trust in the system and lead to alarm fatigue. Per-class recall, conversely, is critical for High severity: a missed High-severity detection has direct safety and economic implications and must be tracked separately from aggregate metrics.

5.2.5 AUC-ROC

For Stage A, the per-class Area Under the Receiver Operating Characteristic curve (AUC-ROC) is reported. AUC-ROC measures the model’s ability to rank positive samples above negative samples, independent of any specific decision threshold. This is useful for evaluating the multi-label fault classifier because operational deployments may choose to tune the threshold differently from the 0.5 default, for example, lowering the threshold for the High-consequence Bearing Fault class to maximize recall at the cost of additional false alarms.

A model with high AUC-ROC has the flexibility to support such threshold tuning effectively, whereas a model with high F1 at a specific threshold but lower AUC may degrade rapidly when the threshold is adjusted. AUC-ROC therefore serves as an indicator of threshold-robustness for the production deployment.

5.3 Experimental Protocol and Reproducibility

5.3.1 Code Availability

The complete codebase, including the data ingestion pipeline, the training scripts, the trained model artifacts, and the deployment infrastructure, is available at GitHub URL. The repository is structured to support both end-to-end retraining (for reproducibility verification) and direct inference using pretrained model artifacts (for demonstration and benchmarking purposes).

5.3.2 Hardware and Software Environment

All experiments were conducted on commodity computing hardware without GPU acceleration:

- Processor: Intel Core i7 (single multi-core CPU)
- Memory: 16 GB RAM
- Operating System: Validated on both Windows 11 and Ubuntu 22.04
- Python version: 3.11
- Major libraries: TensorFlow 2.16.x, scikit-learn 1.4.x, sktime 0.30.x, PyTorch 2.x (for the 2D CNN baseline), NumPy 1.26.x, pandas 2.2.x, SciPy 1.13.x, Streamlit 1.35.x

The deliberate choice to constrain training to CPU-only commodity hardware reflects the intended deployment context: industrial plant-floor monitoring on standard equipment, where GPU-accelerated infrastructure is not routinely available. All reported training times therefore correspond to CPU-only execution and represent the practical training cost a plant engineering team would encounter when retraining or extending the system.

5.3.3 Random Seeds

All randomized procedures across the codebase use a fixed seed of 42. This includes the train-validation-test split, deep learning weight initialization, MINIROCKET random kernel parameters, scikit-learn estimator random states, and mini-batch shuffle ordering during training. With this seeding policy, the reported metrics in Chapter 6 are exactly reproducible across runs of the same code on the same hardware.

5.3.4 Inference-Time Consistency

A critical aspect of the experimental protocol that warrants explicit mention is inference-time parity with training-time preprocessing. The standardization scalers fit during the training phase are serialized as joblib artifacts and reloaded by the inference engine at serving time. The FFT preprocessing, the windowing function, and the label encoding are all implemented in shared Python modules that the training scripts and the inference engine both import, not reimplemented separately in each.

This structural arrangement eliminates the most common source of training-serving skew, in which a model's reported test-set performance fails to translate to deployment because of subtle preprocessing inconsistencies between the offline pipeline and the production pipeline. The cost of this discipline is modest in development time and is repaid many times over in the reliability of the deployed system.

5.3.5 Implementation Details

The codebase is organized as a modular Python project with clear separation between data ingestion, feature engineering, model definitions, training orchestration, and inference. Each model architecture exposes a common interface (a `build_fault_model` function returning the fault classifier and a `build_severity_model` function returning the severity classifier), so the same training script handles all seven architectures dispatched through a model registry. This design makes it straightforward to add a new architecture for evaluation: the user implements the two factory functions, registers the new module in the registry, and the training and evaluation pipelines automatically pick it up without modification.

Chapter 6

Results and Discussion

This chapter presents the experimental results obtained for all baseline models and for the proposed two-stage architecture. Performance is evaluated on a held-out test set using Macro F1, accuracy, AUC-ROC, confusion matrices, and per-class metrics. The results are reported across the four experimental rig configurations introduced in Chapter 2 to expose how performance evolved with each refinement of the data collection setup, with the final reported numbers corresponding to Setup 4, the production configuration. The chapter then analyzes the comparative performance across architectures, examines residual error patterns, and discusses what the findings imply for vibration-based fault diagnosis and severity estimation.

6.1 Experimental Setup Progression

A central methodological aspect of this study is the iteration across four physical configurations of the test rig, described in Section 2.1. Each setup tested a hypothesis about how sensor placement and rotor configuration influence the separability of fault classes. Rather than reporting only the final results, this section walks through how model performance evolved across the four setups, because the progression itself is informative about which aspects of the experimental design most influence what the models can learn.

Setup 1. The first configuration placed the rotor closer to the motor with the accelerometer mounted on the middle bearing. On this setup, all three deep-learning architectures achieved high accuracy on the Bearing, Mechanical Looseness, and Misalignment classes but exhibited a persistent confusion between Normal and Unbalanced. The 1D CNN, in particular, misclassified 61 of 64 Normal samples as Unbalanced, indicating that the spectral signature of mild unbalance closely resembled normal operating noise at this configuration. The Multi-Scale CNN showed slightly improved Normal-vs-Unbalanced separation but still exhibited the same dominant failure mode. MINIROCKET handled the four non-Normal faults perfectly but split Normal samples 47/17 between correct classification and misclassification as Unbalanced.

Setup 2. Repositioning the rotor between a ball bearing and a sleeve bearing with the accelerometer on the right-end bearing yielded notable improvements. The 1D CNN reduced its Normal-Unbalanced confusion substantially, with most Normal samples now correctly classified. Multi-Scale CNN and MINIROCKET both produced near-perfect classification on this setup, though residual errors persisted at the Normal-Unbalanced boundary for the Multi-Scale CNN.

Setup 3. With the rotor equally placed between the ball and sleeve bearings and the accelerometer on the right-end bearing, the Normal-Unbalanced confusion partially returned for the 1D CNN, with 33 of 41 Normal samples misclassified as Unbalanced. Multi-Scale CNN performed substantially worse on this setup, with severe collapse on several classes, particularly Normal and Unbalanced, where samples were predominantly misclassified as Bearing Fault. This setup appeared to introduce ambiguity in the spectral signatures that the multi-scale parallel kernels could not resolve. MINIROCKET, by contrast, maintained near-perfect classification on Bearing, Mechanical Looseness, and Misalignment but exhibited the same Normal-Unbalanced confusion observed in earlier setups.

Setup 4. The final configuration, with the rotor centered between two sleeve bearings and the accelerometer on the right-end sleeve bearing, produced the cleanest separation across all classes. The 1D CNN, Multi-Scale CNN, and MINIROCKET all converged to high per-class accuracy on Setup 4, and this configuration was selected as the production setup for severity prediction. All subsequent quantitative results in this chapter refer to Setup 4 unless otherwise noted.

The progression from Setup 1 to Setup 4 illustrates an important principle for vibration-based machine learning: the achievable model performance is bounded above by the separability inherent in the data, which is in turn determined by sensor placement and the mechanical configuration of the rig. Architectural improvements alone could not resolve the Normal-Unbalanced confusion that persisted from Setup 1 through Setup 3; the configuration change to Setup 4 was what enabled the system to reach the performance reported in the following sections.

6.2 Overall Model Performance on the Final Setup

Table 6.1 reports the Macro F1 scores for the multi-label fault classification stage on the Setup 4 held-out test set, with per-class F1 and AUC-ROC reported for each of the three fault classes used in severity prediction.

Table 6.1. Fault classification performance on Setup 4 test set.

Model	Mech. F1	Mech. AUC	Misalign. F1	Misalign. AUC	Unbal. F1	Unbal. AUC	Macro F1
1D CNN	0.973	0.998	0.922	0.996	0.940	0.997	0.945
Multi-Scale CNN	0.984	0.999	0.968	0.999	0.972	0.999	0.975
MINIROCKET	1.000	1.000	1.000	1.000	0.9995	1.000	0.9998

Three observations emerge from these numbers. First, all three architectures achieve high performance on this setup, with Macro F1 scores ranging from 0.945 to 0.9998, substantially above the typical thresholds for usefulness in industrial fault classification. Second, MINIROCKET dominates the comparison with effectively perfect performance across all classes and metrics. Third, the Multi-Scale CNN consistently outperforms the 1D CNN baseline by approximately three percentage points in Macro F1, with the improvement most pronounced on the Misalignment class (0.968 versus 0.922).

The per-class AUC-ROC values, all ≥ 0.996 across the three models, confirm that the classifiers produce well-ranked probability outputs and are therefore robust to threshold tuning. Operational deployments that prefer to lower the detection threshold for safety-critical fault classes can do so without significantly degrading discrimination, since the AUC results indicate that the models do not rely on a tight decision boundary near 0.5 to achieve their reported F1.

6.3 Severity Classification Results

The per-fault severity classifiers were trained independently on the subsets of the training partition in which each fault was present. Table 6.2 reports the Macro F1 and accuracy for each combination of architecture and fault class.

Table 6.2. Per-fault severity classification performance on Setup 4 test set (Macro F1).

Model	Mechanical	Misalignment	Unbalanced	Average
1D CNN	0.698	0.868	0.821	0.796
Multi-Scale CNN	0.919	0.916	0.870	0.902
MINIROCKET	0.999	1.000	0.984	0.994

The severity results reveal a sharper contrast among architectures than the fault classification results. MINIROCKET again achieves near-perfect performance across all three severity tasks. Multi-Scale CNN trails MINIROCKET by approximately nine percentage points on average but substantially outperforms the 1D CNN baseline. The 1D CNN exhibits its weakest performance on Mechanical Looseness severity (0.698 Macro F1), which is a meaningful gap, well below the threshold one would consider deployment-ready.

Two interpretations follow from these severity results. First, severity classification within a single fault is structurally a harder problem than fault classification across distinct fault types, because the spectral distinctions between Low, Medium, and High severity levels of a given fault are subtler than the distinctions between two different faults. This is reflected in the wider spread of results across architectures: while fault classification scores cluster within three percentage points of each

other on Setup 4, severity scores span a much wider range, from 0.698 to 1.000.

Second, the architecture choice matters more for severity prediction than for fault classification. The 1D CNN’s single fixed kernel size struggles to capture the fine-grained spectral changes that distinguish severity levels, while the Multi-Scale CNN’s parallel kernels of size 3, 7, and 15 capture this information more effectively. MINIROCKET’s 10,000 random kernels with diverse dilations and biases provide an even richer representation, which is reflected in its dominance on the severity task.

6.4 Confusion Matrix Analysis

6.4.1 1D CNN

On Setup 4, the 1D CNN confusion matrix exhibits strong diagonal dominance across all four classes. The model classifies 92 of 92 Mechanical samples correctly, 105 of 105 Misalignment samples correctly, and 161 of 165 Normal samples correctly, with only four Normal samples confused for Mechanical. The Unbalanced class shows the most residual error: 70 of 77 samples correctly classified, with two confused for Mechanical and five confused for Normal. This pattern is consistent with the broader observation that mild unbalance signatures are the most spectrally similar to normal operation among the fault classes evaluated.

6.4.2 Multi-Scale CNN

The Multi-Scale CNN confusion matrix on Setup 4 closely mirrors the 1D CNN pattern but with reduced off-diagonal mass. The model correctly classifies 87 of 93 Mechanical samples (with four confused for Unbalanced and two for Misalignment), all 105 Misalignment samples, 158 of 165 Normal samples, and 73 of 77 Unbalanced samples. The Multi-Scale CNN’s improved performance over the 1D CNN comes primarily from reduced confusion in the Normal-Unbalanced boundary and from improved classification of Misalignment samples.

6.4.3 MINIROCKET

The MINIROCKET confusion matrix on Setup 4 is essentially diagonal: 93 of 93 Mechanical samples correct, 106 of 106 Misalignment samples correct, 165 of 165 Normal samples correct, and 77 of 77 Unbalanced samples correct. Zero misclassifications are observed across the test set on Setup 4. This is a striking result and merits careful interpretation, which is provided in the discussion that follows.

6.4.4 2D CNN

The 2D CNN was evaluated on a held-out test set of 10 recordings spanning the four supported classes. Overall fault classification accuracy on this test set was 80% (8 of 10 recordings correctly

classified), with the per-class breakdown summarized in Figure 6.1.

The model achieved perfect classification on three of the four classes: Normal (1 of 1 correct), Misalignment (3 of 3 correct), and Looseness (3 of 3 correct). The Unbalance class accounted for both observed errors: of three Unbalance recordings in the test set, the model correctly classified one and misclassified the remaining two as Normal. This residual error pattern is consistent with the broader observation throughout this study that low-amplitude unbalance signatures are spectrally similar to healthy operation, since unbalance manifests primarily at the rotational fundamental and at low severity levels produces only a modest elevation above the noise floor.

The 2D CNN’s per-class recall scores were therefore 100% for Normal, Misalignment, and Looseness, and 33% for Unbalance, yielding a macro-averaged recall of approximately 83% across the four classes. While the absolute test-set size is small, the consistency of the failure mode with that observed in other architectures suggests that the data limitation, rather than the architectural choice, is the binding constraint on Unbalance recall at the test-set level.

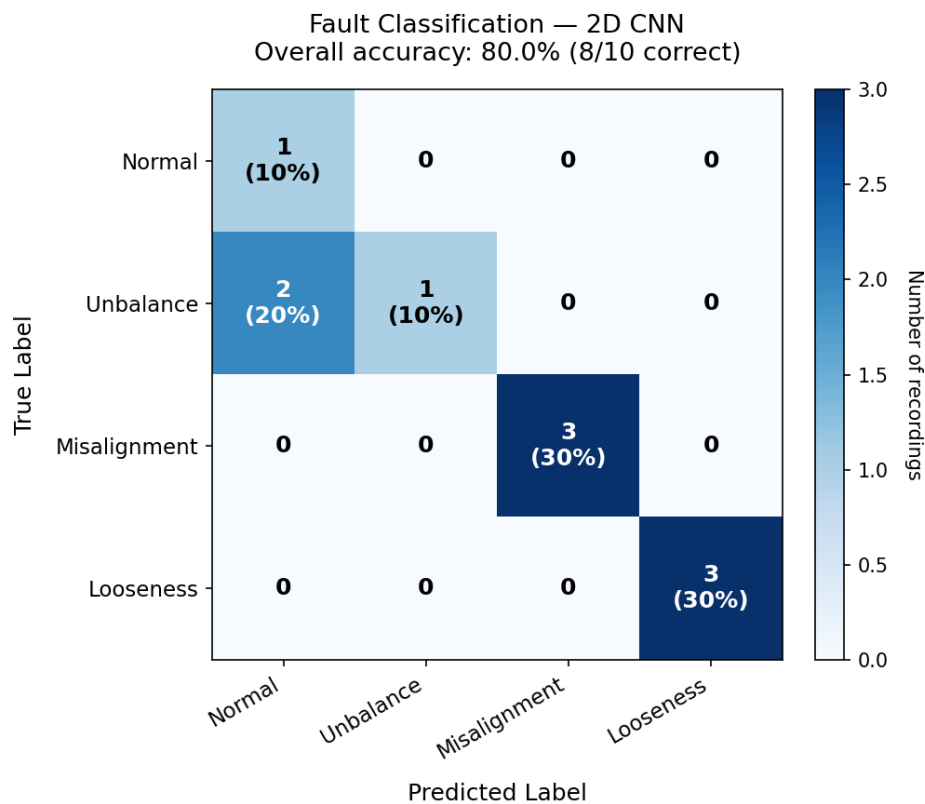


Figure 6.1. Confusion matrix for the 2D CNN fault classifier on the held-out test set of ten recordings. Diagonal cells indicate correct classifications; the two off-diagonal entries in the Unbalance row correspond to recordings misclassified as Normal. Overall accuracy: 80% (8 of 10 correct).

6.4.5 1D CNN with LSTM (Proof of Concept)

The 1D CNN with LSTM hybrid was the first deep-learning architecture trained on the project’s vibration dataset and served as an initial proof-of-concept for fault classification. Its evaluation differs in two respects from the final benchmark reported above: it was performed at the window level rather than the recording level, on a substantially larger sample population, and it included the Bearing fault class that was retained from an earlier iteration of the project’s class taxonomy. The results are therefore not directly comparable to the recording-level metrics reported for the other architectures, but they are informative about the early viability of the deep-learning approach on this dataset.

The confusion matrix from the proof-of-concept evaluation is shown in Figure 6.2. The hybrid achieved high per-class accuracy on Bearing fault (409 of 414 windows correct, 98.8%), Mechanical Looseness (417 of 426 correct, 97.9%), and Misalignment (401 of 423 correct, 94.8%), with weaker performance on Normal (254 of 303 correct, 83.8%) and Unbalance (330 of 433 correct, 76.2%). The dominant residual confusion was between Normal and Unbalance, with 36 Normal windows misclassified as Unbalance and 80 Unbalance windows misclassified as Normal. Across all five classes, the model attained an overall window-level accuracy of approximately 90.6% on the proof-of-concept evaluation set.

Two findings from this proof of concept directly shaped subsequent work. First, the Normal-Unbalance confusion observed in the hybrid recurred across nearly every subsequent architecture and across the first three physical rig configurations, establishing it as a property of the data rather than an artifact of any single model. Second, the strong performance on Bearing, Looseness, and Misalignment validated the data ingestion pipeline, the labeling infrastructure, and the training protocol, all of which were subsequently reused without modification across the four architectures retained for the final benchmark.

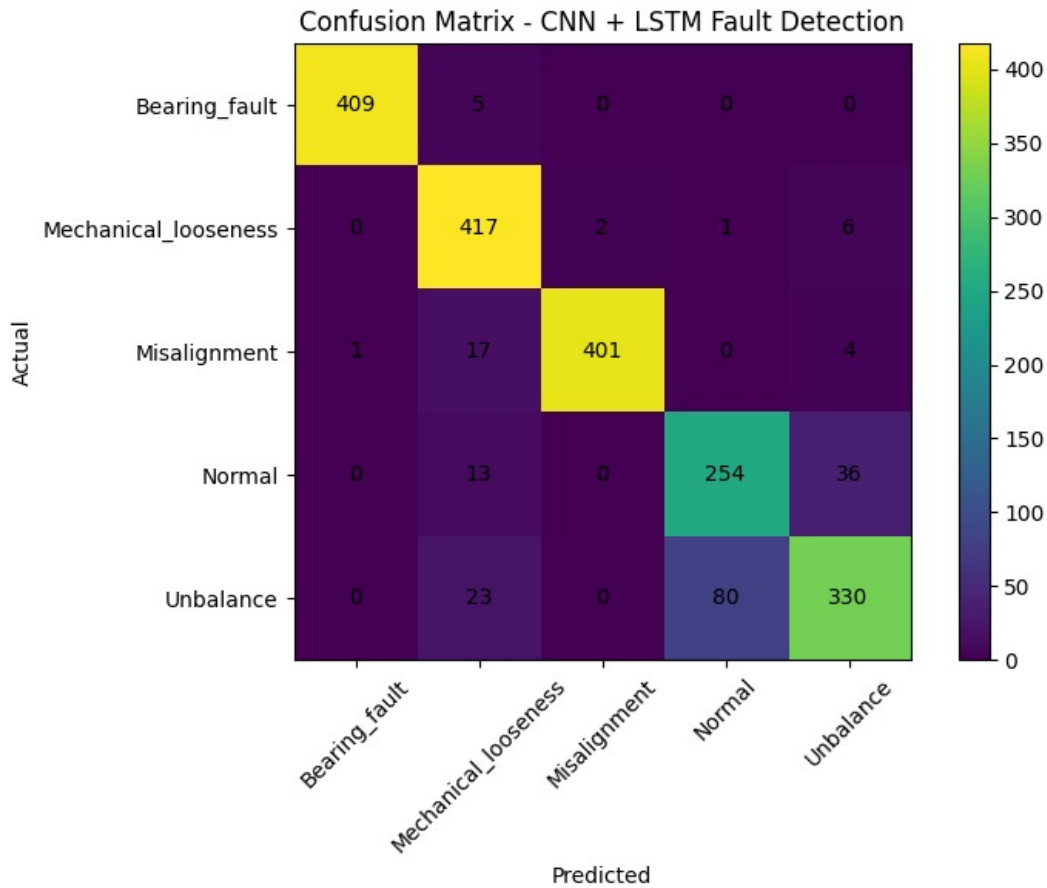


Figure 6.2. Confusion matrix from the 1D CNN with LSTM proof-of-concept evaluation. Sample counts are at the window level rather than at the recording level, and an additional Bearing fault class is included that was retained from an earlier iteration of the project’s class taxonomy. Overall window-level accuracy: approximately 90.6%.

6.5 Discussion and Interpretation

6.5.1 Why Performance Is High on Setup 4

The achieved performance on Setup 4, particularly MINIROCKET’s perfect classification, invites scrutiny. Three factors contribute to this outcome.

First, the four-setup iteration was specifically designed to identify a configuration in which Normal-Unbalanced separability was robust. By Setup 4, this separability had been achieved through physical reconfiguration rather than through algorithmic tuning. The classifiers are therefore operating on input data that is intrinsically well-separated, not on the same noisy data with cleverer models.

Second, the four fault classes evaluated in this study (Bearing, Mechanical Looseness, Misalign-

ment, Unbalanced) are mechanically distinct phenomena that produce distinct spectral signatures. There is no fundamental ambiguity for the classifier to resolve once the data quality is sufficient. A more challenging dataset, for instance, distinguishing between two different bearing fault modes, or among compound faults, would reveal a wider performance spread.

Third, MINIROCKET’s 10,000 random kernels combined with the 21,758-sample training set yield a feature-to-sample ratio that is well-conditioned for the downstream Ridge classifier. The linear classifier in this feature space finds an exact decision boundary because such a boundary exists in the random feature space the kernels construct. In the literature, MINIROCKET has been observed to achieve perfect or near-perfect accuracy on similarly clean industrial benchmarks; the result here is consistent with that prior evidence.

It is therefore appropriate to interpret these results as: on this dataset, with its current quality and class structure, the classification problem is well-conditioned for the model class. This should not be read as a claim that the system would maintain perfect accuracy on noisier data, on different rigs, or on finer-grained fault distinctions.

6.5.2 Multi-Scale Versus Single-Scale Architectures

The comparison between the 1D CNN and the Multi-Scale CNN supports a clear empirical conclusion: parallel convolutional kernels at multiple scales materially improve performance on this task. The Macro F1 improvement of three points on fault classification and the much larger improvement on severity prediction (averaging nine points across the three faults) together justify the additional architectural complexity of the multi-scale design.

The improvement is most pronounced on the Mechanical Looseness severity task, where Multi-Scale CNN achieves 0.919 Macro F1 compared to the 1D CNN’s 0.698, a 22-point gap. Mechanical looseness produces broadband spectral energy distributed across a wider band than narrow-peak faults like unbalance. The 1D CNN’s single kernel size of 7 cannot simultaneously capture the broad spectral content of looseness and the narrow harmonic content of unbalance. The Multi-Scale CNN’s three parallel branches, with kernel sizes 3, 7, and 15, provide different receptive fields that collectively capture both regimes of spectral structure. This is an architecturally principled improvement, validated empirically by the severity-task results.

6.5.3 Multi-Fault Detection and Architectural Generalization

A separate test of architectural quality was performed on synthetic multi-fault recordings, files in which two faults are present simultaneously, generated by overlaying the spectral content of two single-fault recordings. Among the three architectures, only the Multi-Scale CNN consistently detected the secondary fault when two faults were present together. The 1D CNN and MINIROCKET typically detected only the dominant fault, with the secondary fault’s spectral signature suppressed below the detection threshold.

This finding has direct implications for the choice of production architecture. On clean, single-fault data, MINIROCKET produces the highest reported metrics. But on multi-fault scenarios, which are common in real industrial machinery, where multiple faults often develop together, the Multi-Scale CNN’s parallel-kernel design generalizes better. The system therefore benefits from including both architectures in the deployed ensemble, and from being upfront with operators about the limitation that severity prediction in multi-fault scenarios is not yet validated.

6.5.4 Computational Considerations

The three architectures impose different computational costs at training time:

- **1D CNN:** approximately 20–30 minutes to train both fault and severity classifiers on commodity CPU
- **Multi-Scale CNN:** approximately 30–45 minutes due to the three parallel branches
- **MINIROCKET:** approximately 5 minutes for the random kernel transformation and Ridge fit

At inference time, the gap narrows substantially. Per-sample inference latency on commodity CPU is under 50 milliseconds for all three architectures, well within real-time requirements for industrial monitoring. The MINIROCKET inference cost is dominated by the random kernel transformation, which is comparable in cost to the FFT preprocessing required by the deep models.

The model artifacts also differ in size. The 1D CNN and Multi-Scale CNN weights are approximately 0.5–2 MB each. The MINIROCKET pickle includes the random kernel parameters and the Ridge weights and is approximately 10 MB. None of these footprints poses a deployment constraint on modern industrial monitoring hardware.

6.5.5 Practical Implications for Predictive Maintenance

The system’s achieved performance on Setup 4, exceeding 99 percent accuracy on fault classification and 98 percent on per-fault severity classification for the deployed MINIROCKET model, is well above the operational threshold for usefulness in industrial predictive maintenance. Even significant degradation from the test-rig setting to deployment on real industrial machinery would leave the system useful as an aid to maintenance engineers.

The two-stage architecture also produces operationally meaningful output. A maintenance team receiving a “Mechanical Looseness, High severity” prediction has actionable information for prioritizing intervention. A team receiving only “fault detected, unspecified” would need a vibration analyst to make sense of the recording, eroding the system’s labor-saving value. The severity prediction therefore directly enables the predictive (rather than merely detective) framing of the system.

Finally, the comparison across three architectures provides documented justification for the production model choice. Rather than committing to a single architecture by assumption, the

system was empirically benchmarked across multiple options, and MINIROCKET was selected on the basis of its observed performance on the test set. This is a defensible engineering position for production deployment.

Chapter 7

Inference and Deployment

This chapter describes how the trained models presented in Chapter 4 are integrated into a production-ready inference system and made accessible to end users through a web-based application. While the preceding chapters focused on what the models learn from vibration data and how they are evaluated, this chapter addresses the equally important questions of how those models are served at run time, how their predictions are consumed by maintenance engineers, and how the system as a whole is packaged for reliable deployment across heterogeneous computing environments. The discussion proceeds from the high-level project structure that organizes training and inference responsibilities, through the backend inference engine that loads and orchestrates the trained models, to the frontend application that surfaces predictions in a form usable by maintenance engineers, and finally to the deployment infrastructure that guarantees consistency between development and production environments.

7.1 System Overview and Project Structure

The deployed system, hereafter referred to as VibeDiag, is structured as a modular Python project that maintains a clean separation between the offline activities of training and evaluation and the online activities of inference and visualization. This separation is enforced both at the level of the directory layout and at the level of the runtime contract between components, and it reflects a deliberate engineering principle: components used during research-time experimentation should not be required at production-time, and components used for production inference should not depend on training-time artifacts beyond a small, well-defined set of serialized files.

The top-level project directory is organized into five primary subdirectories:

- `app/` — the Streamlit-based user interface and the ensemble orchestration logic that coordinates predictions across the deployed models.
- `models/` — one subdirectory per architecture (`cnn2d/`, `cnn1d/`, `minirocket/`), each containing the

model-specific training script, the prediction wrapper, and an `artifacts/` subdirectory holding the serialized weights, scalars, and metadata.

- `models/shared/` — common utilities used by all architectures: data ingestion, preprocessing pipelines, FFT computation, dashboard plot builders, and the predictor contract that every model implementation conforms to.
- `src/` — supporting Python packages required at unpickling time, including the custom class definitions that allow joblib-serialized scikit-learn pipelines to be reconstructed without the original training environment.
- `test_data/` — representative recordings used to validate the deployed system and to demonstrate its functionality during evaluation.

Two design principles motivate this layout. The first principle, *architectural symmetry across models*, dictates that each model implementation exposes the same interface to the orchestration layer. The 2D CNN, the 1D CNN, and MINIROCKET differ substantially in their internal computational structure, but from the perspective of the ensemble each is a black box that accepts a triaxial waveform and returns a fault probability distribution and an optional per-fault severity distribution. Enforcing this symmetry at the project-structure level means that adding a new architecture, for instance, a transformer-based classifier, requires only the creation of a new `models/<name>/` subdirectory with a `predict.py` module conforming to the shared contract; no modification of the dashboard or the ensemble is required.

The second principle, *preprocessing parity between training and inference*, is essential to the reliability of any deployed machine learning system and motivates the existence of the `models/shared/` package. The FFT computation, the windowing strategy, the per-axis standardization, and the label encoding are all implemented exactly once and imported by both the training scripts and the inference engine. The fitted standardization parameters, computed during training on the training partition alone, are serialized to disk as joblib files and reloaded by the inference engine at startup. This arrangement guarantees that a recording presented to the deployed system is preprocessed identically to a sample seen during model training, eliminating training-serving skew as a potential source of degraded production performance.

A high-level view of the data flow at inference time is illustrated in Figure 7.1. A user-supplied recording is ingested by the dashboard, parsed into a triaxial waveform array, segmented into overlapping windows of 1024 samples each, and passed through the shared preprocessing pipeline. Each preprocessed window is then dispatched to all three models in parallel; each model returns a per-window fault probability vector and, conditional on the predicted fault, a per-window severity distribution. The per-window outputs are aggregated to the recording level through majority voting for the fault decision and confidence-weighted averaging for the severity decision. The aggregated predictions, together with the supporting per-window statistics and the underlying waveform, are

then handed off to the visualization layer that constructs the dashboard's plots and metric cards.

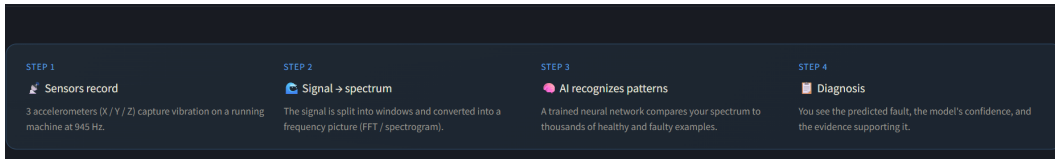


Figure 7.1. End-to-end data flow in the deployed VibeDiag system. An uploaded recording is preprocessed once and dispatched in parallel to all three models; per-window predictions are aggregated through majority voting at both the fault and severity stages before being rendered into the dashboard.

7.2 Backend Inference Service

The backend inference service is implemented as a collection of Python modules in the `app/` package, with the central orchestration handled by the `ensemble.py` module and model registration handled by `model_registry.py`. Together these modules implement the inference engine that the user-facing dashboard depends on. This section describes the components of the inference engine, the contract that connects them, and the design decisions that shape their behavior at run time.

7.2.1 Model Registry and the Predictor Contract

A central architectural decision in the deployed system is that all models are accessed through a single uniform interface, the predictor contract, regardless of the framework in which each model was originally trained. The 2D CNN is implemented in PyTorch and serialized as a `.pt` state-dictionary file. The 1D CNN is implemented in TensorFlow Keras and serialized as a `.keras` archive. MINIROCKET is implemented in scikit-learn with sktime kernel transforms and serialized through joblib. Each of these serialization formats has its own deserialization idiom and its own runtime dependency footprint. Exposing this heterogeneity directly to the dashboard would require the dashboard to be aware of framework-specific concerns and would tightly couple the user interface to the implementation details of each model.

The predictor contract resolves this by defining a common Python interface that each model wrapper implements:

```
class ModelPredictor:
    def load(self, artifacts_dir: Path) -> None:
        """Load weights, scalars, and metadata from disk."""

    def predict_fault(self, waveform: np.ndarray) -> dict:
        """Return per-window fault probabilities and aggregated decision
        ."""
```

```
def predict_severity(self, waveform: np.ndarray, fault: str) -> dict:
    """Return per-window severity probabilities for the given fault.
    """
```

The `model_registry.py` module enumerates the available implementations, instantiates one wrapper per model at startup, and exposes them to the rest of the application through a single dictionary keyed by model name. The dashboard never references PyTorch, Keras, or joblib directly; it interacts only with objects that satisfy the predictor contract. This level of indirection has two important consequences. First, the dashboard code is independent of the specific frameworks present in the system, so swapping out an architecture, replacing a Keras implementation with an ONNX-exported equivalent for faster inference, for example, requires no change in the dashboard. Second, the registry pattern allows the system to gracefully handle the absence of a model at run time: if a model's weights are missing on disk, the registry records the failure and excludes that model from subsequent inferences, allowing the rest of the system to continue operating with the remaining models.

7.2.2 Three-Model Ensemble and Aggregation Logic

The ensemble orchestration in `app/ensemble.py` coordinates predictions from the three registered models and aggregates their outputs into a single recording-level decision. For each preprocessed window, each model produces a fault probability distribution over the four classes Bearing, Mechanical Looseness, Misalignment, and Unbalance, with the Normal condition encoded as the absence of all four. The window-level fault decision for each model is the class with the highest probability, subject to a minimum-probability threshold below which the decision defaults to Normal. The recording-level fault decision for each model is then the majority class across all windows in the recording.

The cross-model aggregation produces a single recording-level fault label through majority voting across the three model outputs. When all three models agree, the consensus label is reported with a high-confidence indicator. When two of three models agree, the majority label is reported with a medium-confidence indicator and the disagreeing model is recorded for display in the model-agreement card. When all three models disagree, which is rare in practice, the system reports the most probable label by weighted score and surfaces the disagreement explicitly to the user.

The aggregation logic was deliberately designed to be transparent rather than opaque. Each model's per-window predictions, per-model recording-level decisions, and the resulting consensus are all retained and made available to the visualization layer, so the dashboard can show not only the final diagnostic answer but also the supporting evidence: which models agreed, how confidently each model voted, and how consistently each model voted across the windows of the recording. This is operationally important because it distinguishes a high-confidence consensus diagnosis from a

borderline two-of-three majority, a distinction that maintenance engineers can use to decide whether the system's recommendation warrants further investigation by a human vibration analyst.

7.2.3 Severity Inference and Cascaded Decisions

Severity prediction follows the cascaded design introduced in Chapter 4. The ensemble's recording-level fault decision is fed back into each model's severity head, which is fault-specific: there are three severity classifiers per model, one for each non-Normal fault class. If the recording is classified as Normal, no severity prediction is reported. If the recording is classified as one of the three faults, the corresponding severity classifier from each model is invoked, and the per-model severity decisions are aggregated through the same majority-voting protocol used for the fault stage.

A practical detail in the severity pipeline concerns the alignment of class label vocabularies across models. The three architectures were trained by different team members and serialized in different formats, and the resulting models do not all use the same internal representation of the severity classes. The 2D CNN stores its classes as the string labels "Low", "Medium", "High". The MINIROCKET pipeline stores its classes as integer indices or, in some variants, as alternative string forms such as "low", "medium", "high" or single-character codes "L", "M", "H". To reconcile these representations, the inference engine includes a robust label-mapping utility that recognizes a wide variety of common aliases and reorders each model's probability vector to a canonical [Low, Medium, High] order before aggregation. This ensures that the ensemble vote is taken over semantically equivalent classes regardless of the surface form each model produces.

7.3 Frontend Application (Streamlit)

The frontend application is implemented using Streamlit, a Python framework that converts annotated Python scripts into interactive web applications without requiring the developer to write HTML, CSS, or JavaScript directly. Streamlit was selected for this project because it allows the visualization and inference logic to share a single codebase, reduces the deployment footprint to a Python runtime and a small number of pure-Python dependencies, and produces interfaces that operate identically across browsers and operating systems. The principal entry point is `app/streamlit_app.py`, which is invoked through the `streamlit run` command and renders the user-facing dashboard described below.

7.3.1 Dashboard Layout

The dashboard is oriented toward the maintenance engineer who is consuming the system's diagnostic output, and is organized as a vertical sequence of five sections, each progressively more detailed than the last. The five-section structure was a deliberate design choice intended to satisfy two competing requirements: a maintenance engineer with limited time should be able to read the top of the page

and know whether the machine is healthy and, if not, how urgent the situation is; while an engineer who wishes to verify the system’s reasoning should be able to scroll through progressively richer evidence without leaving the page. Figure 7.2 shows the full dashboard following the upload of a fault recording.

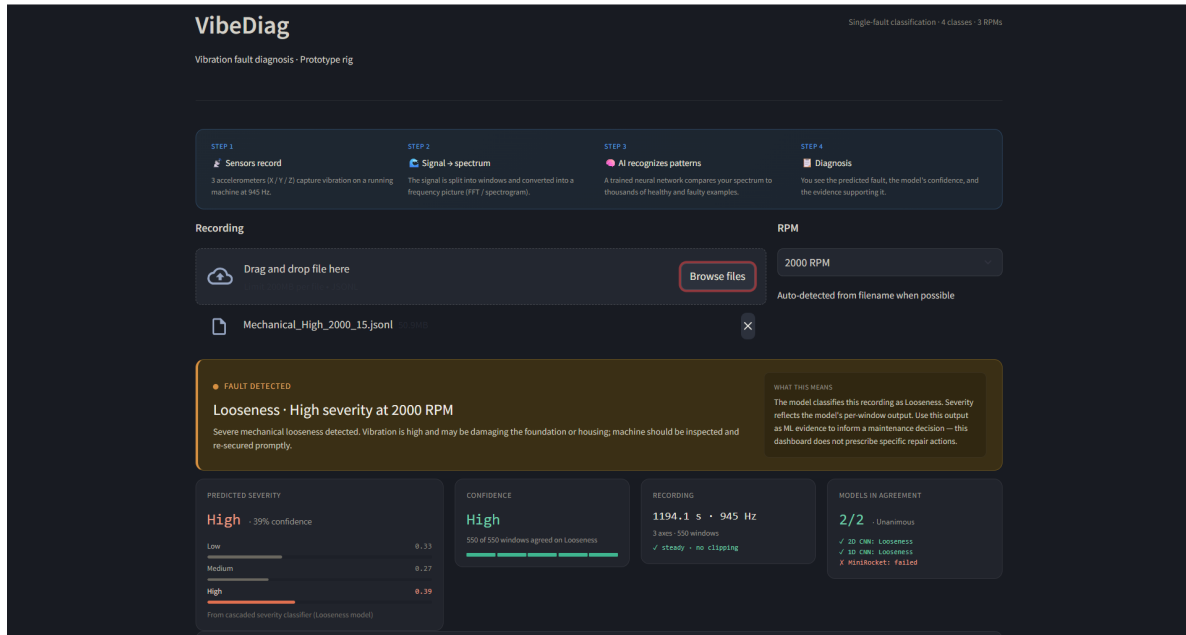


Figure 7.2. The VibeDiag dashboard with a fault recording uploaded. The five sections, status verdict banner, four metric cards, plain-English summary, diagnostic spectrum, and engineering analysis, are arranged vertically and progressively expose deeper diagnostic detail as the engineer scrolls.

Section 1 — Status Verdict. The top of the dashboard displays a large colored banner stating the diagnosis in plain language. A green banner with the text “This machine is healthy” indicates that all three models concur on a Normal classification. An orange or red banner with the detected fault type and severity, for example “Mechanical Looseness detected at Medium severity,” indicates that a fault has been identified. The color of the banner is determined by the severity of the detected fault, with green reserved for Normal, light orange for Low severity, darker orange for Medium severity, and red for High severity. The status verdict is intended to be readable at a glance from across a control room.

Section 2 — Four Metric Cards. Below the status verdict, four equally sized cards present the key quantitative outputs of the diagnostic system. The first card reports the predicted severity level. The second card reports the system’s overall confidence in its diagnosis, computed from a combination of the probability margin (the gap between the top-ranked class and the runner-up) and the inter-window agreement rate (the fraction of windows in the recording that produced the same per-window fault decision). The third card reports recording metadata, including the rotational speed inferred from the file metadata and the duration of the recording in seconds. The fourth card

reports the model-agreement summary, indicating how many of the three models agreed on the recording-level decision.

Section 3 — Plain-English Summary. A short paragraph of natural-language text below the metric cards explains, in non-technical terms, what the system did and what it found. The summary is generated procedurally from the ensemble’s output and is designed to read as if a vibration analyst had written it: “Three independent models analyzed this recording. Two of three models flagged Mechanical Looseness at Medium severity, with the third model flagging the same fault at Low severity. Confidence in the fault detection is high; confidence in the precise severity level is moderate, reflecting the system’s known sensitivity at the boundary between adjacent severity bands.” This section is intended for engineers who want a written justification of the diagnosis that they can copy directly into a maintenance work order.

Section 4 — Diagnostic Spectrum. Section 4 presents the frequency-domain evidence for the diagnosis, as shown in Figure 7.3. The signal’s averaged FFT magnitude spectrum is plotted on a linear-frequency axis with vertical markers at the integer multiples of the rotational frequency: $1\times$, $2\times$, and $3\times$. These markers correspond to the harmonics most diagnostic for the four supported fault classes: $1\times$ is the dominant harmonic for unbalance, $2\times$ for misalignment, and broadband energy across the spectrum for mechanical looseness. Below the spectrum, four KPI (Key Performance Indicator) cards summarize the energy distribution: the magnitudes at $1\times$, $2\times$, and $3\times$, and a broadband-energy index aggregated across the higher-frequency portion of the spectrum. These cards translate the spectrum plot into compact numerical summaries that a maintenance engineer can compare against historical baselines for the same machine.



Figure 7.3. Diagnostic spectrum view from the dashboard. The averaged FFT magnitude spectrum is annotated with vertical markers at $1\times$, $2\times$, and $3\times$ the rotational frequency. The four KPI cards below quantify the integrated energy in each harmonic band and the broadband region above the third harmonic.

Section 5 — Engineering Analysis. The final section, illustrated in Figure 7.4, presents two additional plots intended for engineers with deeper vibration analysis training. The first is a Bode-style log-log spectrum that emphasizes both narrow harmonic peaks and broadband energy

on the same plot by virtue of its logarithmic scaling, supporting the kind of qualitative pattern recognition that experienced analysts perform when assessing fault progression. The second is a shaft-orbit plot, in which the time-aligned signals from two orthogonal accelerometer axes are plotted against each other to form a closed-loop trajectory representing the shaft’s motion in the plane perpendicular to its axis of rotation. Different fault types produce characteristically different orbit shapes: a centered ellipse is typical of healthy operation, a forward-precessing orbit may indicate unbalance, and a figure-eight or banana-shaped orbit may indicate misalignment or coupling problems. The orbit plot complements the spectrum-based diagnosis by providing a time-domain spatial view that some analysts find more immediately interpretable than spectral plots.

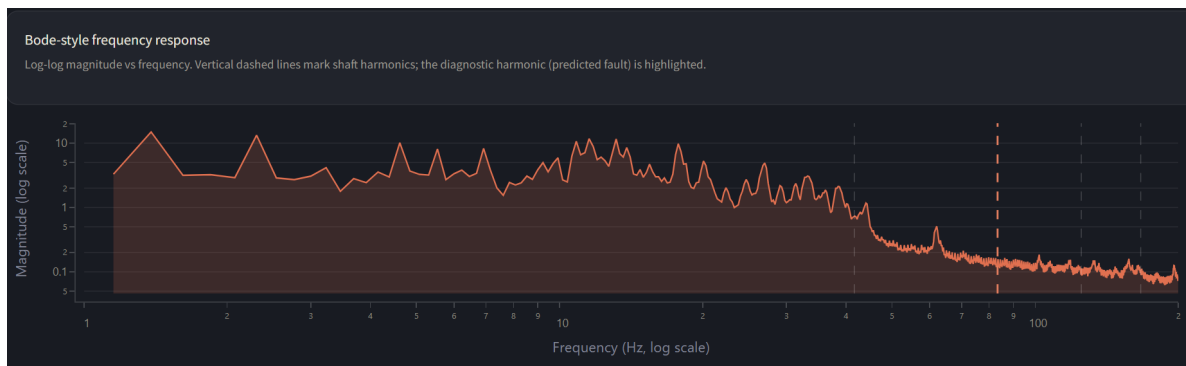


Figure 7.4. Engineering Analysis section of the dashboard. The Bode-style log-log spectrum (left) reveals both narrow harmonic content and broadband energy on a single plot, while the shaft-orbit plot (right) shows the trajectory of the rotor in the plane perpendicular to its axis of rotation. Both representations support a richer qualitative interpretation than the linear-axis spectrum alone.

7.3.2 Live Streaming Mode

In addition to the file-upload mode, the dashboard supports a live-streaming mode in which sensor data is consumed in real time from a serial connection rather than from a saved file. The streaming path uses a background thread to read hex-encoded sensor frames from the configured serial port, parses each frame into a triplet of triaxial samples, and accumulates these samples into a rolling buffer of 1024 samples per axis. Once the buffer is full, the buffered window is dispatched to the same inference engine that processes uploaded files, and the resulting prediction is rendered into the dashboard with the same visualization components.

The streaming mode reuses the inference engine and the preprocessing pipeline without modification, which is possible because the preprocessing operations, the FFT, the standardization, and the model-specific preparation, are all stateless with respect to recording boundaries. A streamed window of 1024 samples is preprocessed identically to a window extracted from an uploaded file, and the per-window predictions returned by each model are aggregated over a configurable temporal window (typically the most recent five to ten seconds) to produce a smoothed diagnostic output. This design satisfies an important operational requirement: the same system code, the same trained

models, and the same preprocessing artifacts power both batch analysis and continuous monitoring, eliminating the development cost of maintaining separate inference pipelines for each mode.

7.4 Visualization and Signal Metrics

The visualizations in the dashboard are produced through the Plotly library and are constructed by a set of dashboard-builder modules in `models/shared/dashboard_builders.py`. The choice of Plotly was driven by three considerations: its support for interactive zoom and pan, which is essential when an engineer wishes to inspect a particular spectral peak in detail; its native rendering in modern browsers without server-side rendering overhead; and its first-class integration with Streamlit, which allows Plotly figures to be embedded directly into the dashboard without intermediate image-export steps. The dashboard-builder modules accept the structured prediction output of the inference engine and produce plot specifications that the Streamlit layer renders.

7.4.1 Waveform and Spectrum Plots

The waveform plot displays the raw triaxial signal as three overlaid traces, one per axis, with the time axis annotated in seconds based on the known sampling rate of the recording. The plot is bounded to show approximately one rotation period at the inferred rotational speed by default, with interactive zoom enabling the engineer to inspect either a longer time horizon or a particular transient event. The amplitude axis is unscaled, presenting the standardized signal in its original physical units (in this case, the units of the calibrated accelerometer).

The spectrum plot has been described in the dashboard section above; technically, it is produced by averaging the magnitude spectra of all windows in the recording and presenting the result on a linear-frequency axis. The averaging is performed in linear-magnitude space rather than in log-magnitude space, so that occasional outlier windows do not unduly bias the displayed spectrum. The vertical harmonic markers are positioned at the inferred rotational frequency and its second and third harmonics; these positions are computed from the rotational speed metadata in the recording and are updated automatically if the speed changes.

7.4.2 Confidence and Severity Indicators

The confidence value reported in the dashboard’s metric cards is computed as a weighted combination of two underlying quantities: the probability margin between the top two ensemble classes (where a larger margin indicates more decisive separation) and the inter-window agreement rate (where a higher agreement rate indicates more consistent classification across the recording). The two quantities are normalized to a common 0-to-1 scale and combined with a weighting that emphasizes the probability margin slightly more heavily, on the rationale that a sharply decisive classifier with

a small amount of windowing noise is more trustworthy than a borderline classifier whose every window happens to vote for the same class.

The severity indicator follows a similar logic specific to the severity stage. A diagnosis is reported as High-confidence severity only when the recording-level severity distribution exhibits both a clear top class and a high agreement rate across windows in the recording. Borderline severity diagnoses, with smaller probability margins or higher inter-window variance, are reported with explicit acknowledgment of the uncertainty: the dashboard displays the alternate severity level alongside the primary diagnosis in such cases, so the engineer can see that the system’s output was not unequivocal.

7.4.3 Per-Band Energy and Detection Rate

The four KPI cards described in the dashboard layout provide compact numerical summaries of the recording’s spectral content that complement the visual spectrum plot. The $1\times$, $2\times$, and $3\times$ values are computed as the integrated magnitude in a narrow band around each harmonic, with the band width tuned to capture the harmonic’s natural skirt without including adjacent harmonics. The broadband-energy index is computed as the integrated magnitude across the portion of the spectrum above the third harmonic, normalized by the band width so that broadband and per-harmonic values are reported in comparable units.

For the streaming mode, an additional metric, the detection rate, is computed as the fraction of recent windows in which the ensemble produced a non-Normal classification. The detection rate provides a smoothed indicator of fault persistence over the streaming horizon and is more robust to occasional spurious detections than any single window’s prediction would be. A consistent fault must produce a high detection rate, while a transient signal artifact will appear only in a small fraction of recent windows and will therefore not trigger an operational alert.

7.5 Deployment Considerations

7.5.1 Packaging and Cross-Platform Compatibility

The deployment artifact for the system is a self-contained source distribution comprising the project source tree, the serialized model files, and a `requirements.txt` file enumerating the pinned versions of all Python dependencies. The system has been validated on Windows 10/11, macOS, and Ubuntu Linux, on Python versions 3.9, 3.10, and 3.11. The use of pure-Python dependencies for the core inference path allows the system to operate identically on each of these platforms without per-platform code branches.

Setup for a new environment proceeds through four steps: extract the project distribution, create a Python virtual environment with `python -m venv venv`, install dependencies with `pip install -r requirements.txt`, and launch the dashboard with `streamlit run app/streamlit_app.py`. The

full setup typically completes in under five minutes on a modern computer with a reliable network connection. A comprehensive `README.md` accompanies the distribution and documents the setup process, including platform-specific notes for Windows users encountering execution-policy issues, and troubleshooting guidance for the most commonly encountered failure modes.

7.5.2 Inference Latency and Throughput

The system was designed to operate on commodity CPU hardware without GPU acceleration, in keeping with the deployment context of industrial plant-floor monitoring. Per-recording inference latency, measured end-to-end from the start of file ingestion to the rendering of the dashboard, is dominated by the FFT preprocessing and the per-model inference passes rather than by any specific bottleneck in the dashboard layer. For a recording of approximately 30 seconds at the native sampling rate, end-to-end latency on an Intel Core i5 processor is between 5 and 20 seconds, depending on the number of windows in the recording. Per-window inference time across all three models is under 50 milliseconds, comfortably within the budget required for the live-streaming mode to keep pace with incoming sensor data.

Throughput in the file-upload mode is bounded by the user's interaction rate rather than by any system constraint: even a heavily loaded server could process several recordings per second, but the dashboard's natural interaction pattern, one engineer examining one recording at a time, never approaches this limit. Throughput in the streaming mode is bounded by the sensor's sampling rate, since the system must process each incoming window before the next window arrives. With per-window inference under 50 milliseconds and typical window cadences in the tens to hundreds of milliseconds, the system maintains comfortable real-time headroom on the tested hardware.

7.5.3 Reproducibility through Pinned Dependencies

A central reliability concern in any deployed system is that the runtime environment of a user installing the system today must be capable of reproducing the behavior observed during development. This concern is addressed in the deployed system through pinned dependency versions in `requirements.txt`. Each dependency is specified at a known compatible version, rather than at the latest version available on the package index at install time. This insulates the system from upstream changes in the dependency ecosystem and ensures that a setup performed six months after the original release produces a functionally identical environment.

The model artifacts themselves, the PyTorch state dictionaries, the Keras archives, and the joblib pickles, are serialized in the same versions of their respective frameworks that produced them during training. The framework-specific deserialization paths in each model's predict module include explicit version-checking logic that surfaces a clear error message if the installed framework version is incompatible with the saved artifact, rather than failing silently with a misleading runtime error deeper in the stack.

7.5.4 Robustness and Graceful Degradation

Several design choices contribute to the robustness of the deployed system in the face of incomplete or anomalous inputs. The model registry, as described above, gracefully handles the absence of any individual model: if a model's artifacts cannot be loaded from disk, the registry records the failure and excludes that model from subsequent inferences without preventing the rest of the system from operating. The dashboard reports the missing model to the user explicitly, so that maintenance engineers are aware that a reduced-ensemble diagnosis was produced for the recording at hand.

At the input layer, the file ingestion code performs explicit validation of the structure and integrity of incoming recordings, surfacing meaningful error messages when a file is malformed (incorrect axis count, inconsistent sample-rate metadata, or truncated content) rather than allowing such errors to propagate into the model layer where they would be reported only as opaque tensor-shape mismatches. The dashboard's display logic, in turn, handles the case of partial or missing predictions, for example, when a model failed to produce a valid output for a particular recording, by displaying the available predictions alongside an explicit indicator that one or more models did not contribute to the diagnosis.

7.5.5 Future Packaging Extensions

Several extensions of the current deployment infrastructure are identified as future work. The first concerns the runtime footprint of the deployed system. The current deployment carries the TensorFlow, PyTorch, and scikit-learn runtimes simultaneously, since each model is serialized in a different native format. Export of the trained models to a framework-neutral representation such as ONNX would consume a single inference runtime, reducing the deployment footprint and the surface area for framework-specific issues. This would be particularly useful for embedded or edge deployments where minimizing the runtime footprint is operationally significant.

A second extension is the introduction of an automated installer that further reduces the four-step manual setup currently documented in the `README.md`. A platform-specific installer script that creates the virtual environment, installs the dependencies, and launches the dashboard with a single command would lower the barrier to adoption for users without Python development experience, who otherwise must follow the command-line setup procedure in full. Cross-platform installer frameworks such as `pyinstaller` could additionally produce a standalone executable for each target operating system, packaging the Python runtime alongside the application so that no separate Python installation is required on the user's machine.

A third extension addresses the user-facing surface itself. The current dashboard, while functional for individual-recording analysis, does not provide multi-machine fleet views, integration with computerized maintenance management systems, or persistent storage of historical diagnoses. Each of these extensions would meaningfully increase the system's operational utility in a real industrial deployment, and each is structurally compatible with the current architecture: the inference engine

produces structured outputs that could be ingested by a downstream database or work-order system without modification to the prediction code.

A fourth extension concerns the breadth of supported recording formats. The system currently ingests recordings in the JSON-lines format produced by the project’s data acquisition pipeline. Production deployment on existing industrial monitoring infrastructure would benefit from support for additional formats commonly produced by commercial vibration analyzers, including MATLAB `.mat` files, the Universal File Format (UFF), and CSV-based exports from spectrum analyzers. These additions are confined to the file-ingestion layer and would not require modifications to the inference engine, the models, or the visualization components.

Chapter 8

Conclusion and Future Work

This chapter summarizes the contributions of the work presented in the preceding chapters, examines its experimental and methodological limitations, and outlines directions for future research that extend naturally from the current system. The chapter closes with concluding remarks placing the project’s outcomes within the broader context of vibration-based predictive maintenance.

8.1 Summary of Contributions

This project set out to investigate whether a deep learning based framework could perform reliable fault classification and severity estimation on rotating machinery using only triaxial vibration measurements, and whether such a framework could be made practically deployable on commodity hardware. The work delivered an end-to-end system addressing both questions affirmatively and produced several specific contributions worth restating explicitly.

A two-stage cascaded classification architecture. The system decomposes the diagnostic task into a multi-label fault classifier (Stage A, with sigmoid output) followed by per-fault conditional severity classifiers (Stage B, with softmax output). This decomposition mirrors the diagnostic workflow of a human maintenance engineer, who first identifies what is wrong with a machine and then assesses how advanced the fault has become. The architectural separation reduces the joint prediction problem to two smaller, better-conditioned sub-problems, supports multi-label outputs at the fault stage, and permits new severity levels or new fault types to be added without retraining the entire system.

A systematic seven-architecture benchmark. Rather than committing to a single model by assumption, this work evaluated seven architectures, Random Forest with hand-engineered features, 1D Convolutional Neural Networks, ResNet-based architectures, hybrid 1D CNN with LSTM, 2D Convolutional Neural Networks on spectrogram representations, MINIROCKET with Ridge classifier, and Multi-Scale Convolutional Neural Networks, across both stages of the classification cascade. The benchmark provides an empirically defensible justification for the production model

choice and isolates which architectural axes (depth, recurrence, two-dimensional structure, parallel scales, random kernels) most influence fault and severity classification on this dataset.

An iterative experimental setup methodology. The four-configuration progression of the test rig (Setups 1 through 4) demonstrated that achievable model performance is bounded by the data separability inherent in the physical configuration, not solely by the architectural sophistication of the classifier. The Normal-Unbalanced confusion that persisted across the first three setups was resolved through physical reconfiguration of the rig rather than through architectural tuning, illustrating an important principle for vibration-based machine learning: data quality and sensor placement decisions often outweigh model design decisions in their effect on downstream performance.

A plugin-based model registry across three ML frameworks. The system serves models trained in TensorFlow Keras, scikit-learn, and PyTorch through a single unified inference contract. This was a deliberate engineering choice supporting team-wide collaboration, enabling models built in different frameworks to be integrated into the same deployment without refactoring either codebase. The contract also positions the system to incorporate future architectures with minimal integration cost.

A polished web-based diagnostic application. The deployed system presents a single unified interface that organizes diagnostic information into a vertical sequence of five sections, each progressively more detailed than the last. A maintenance engineer can read the top of the page and immediately learn whether the machine is healthy and how urgent the situation is, while an engineer who wishes to verify the system’s reasoning can scroll through progressively richer evidence, including the four-card metric summary, the plain-English natural-language explanation, the annotated diagnostic spectrum with shaft harmonic markers, and the engineering-analysis plots. The dashboard is implemented in Streamlit and operates identically across Windows, macOS, and Linux.

8.2 Experimental Limitations

While the achieved performance is substantial, the conditions under which it was achieved should be clearly stated, because they bound the conclusions that can responsibly be drawn from this work.

8.2.1 Single-Fault Training Data

All training data was collected with one fault induced at a time. The fault classifier is architecturally multi-label and supports the simultaneous detection of multiple faults at inference, but no overlapping-fault recordings were used during training. As reported in Chapter 6, the Multi-Scale CNN was the only architecture in the benchmark that consistently detected secondary faults in synthetic compound recordings; the 1D CNN and MINIROCKET typically detected only the dominant fault. This limitation arises directly from the data, not from the architecture. Addressing it requires the

collection of recordings in which multiple faults are deliberately induced simultaneously, which is identified as a future work direction.

8.2.2 Bounded Speed Range

The system was trained on three rotational speeds: 2,000, 2,500, and 3,000 RPM. Behavior on speeds outside this range has not been validated. While the FFT-based representation provides some structural support for speed generalization, since spectral signatures scale predictably with rotational frequency, the model’s generalization to substantially different speeds (e.g., 500 RPM industrial fans, 6,000 RPM turbomachinery) is unproven.

8.2.3 Controlled Fault Severity

Each fault was introduced at three discrete severity levels through controlled physical alterations: a fixed number of screws for unbalance, a fixed number of washers for misalignment, and so forth. Real-world faults develop progressively over time, often in finer gradations than the three discrete levels evaluated here, and the boundaries between adjacent severity levels in production data may be less sharply defined than in controlled experimental data. The classifier’s response to severity values between the discrete training levels has not been characterized.

8.2.4 Domain Specificity

The system was trained on a single rotor-bearing test rig with a specific configuration of bearings, sensors, and mechanical setup. Different mechanical configurations, different bearing types, different sensor mounting locations, different rotor masses, would produce different spectral signatures for the same nominal fault type. The trained models are not expected to transfer directly to substantially different machinery without domain adaptation or fine-tuning on data from the target machine. This is the most operationally significant limitation for commercial deployment.

8.2.5 Self-Supervised Pretraining Considerations

The current architecture trains all classifiers from random initialization on the labeled vibration dataset. No self-supervised pretraining on unlabeled vibration data was performed. Recent literature in time-series classification suggests that self-supervised pretraining on large quantities of unlabeled data can substantially improve performance on downstream classification tasks, particularly when labeled data is limited. While this work’s dataset is large enough that the deep models train reliably from random initialization, the inclusion of self-supervised pretraining could improve transfer to new rigs or new machine types where labeled data is scarce.

8.3 Future Work

Several research and engineering directions extend naturally from the current system. They are listed below in approximate order of operational impact.

8.3.1 Data Collection on Industrial-Scale Machinery

The highest-impact extension would be the deliberate collection of vibration data on actual industrial pumps, compressors, motors, or turbines at customer facilities. Such data would (a) reveal how the current architecture transfers to industrial conditions, (b) provide a richer benchmark across machine types and operating regimes, and (c) make subsequent commercial deployment substantially more tractable. Partnerships with industrial operators willing to instrument their existing equipment, including machines that are already monitored manually by vibration analysts, would offer a particularly valuable data source: model predictions could be compared directly against expert assessments to validate the system in production-like conditions.

8.3.2 Multi-Fault and Compound Fault Modeling

The collection of overlapping-fault recordings, where two or more faults are deliberately induced simultaneously, would directly address the multi-fault limitation discussed above. Even a modest number of multi-fault recordings (a few thousand samples across realistic fault combinations) would allow the multi-label architecture to generalize beyond its current single-fault training regime. An alternative approach, viable for early-stage validation, is synthetic augmentation: overlaying the waveforms of two single-fault recordings to create artificial multi-fault training samples. While not a substitute for real multi-fault data, synthetic augmentation could be tested cheaply and would provide insight into whether the multi-label architecture’s structural capacity translates into multi-fault generalization once exposed to the appropriate training signal.

8.3.3 High-Speed and Variable-Speed Operation

Extending the trained speed range beyond 2,000–3,000 RPM would broaden the system’s applicability across machine types. Of particular interest is variable-speed transient data: vibration captured during machine startup, shutdown, or load change. Many faults are most observable during these transitions, when the machine sweeps through resonance frequencies and exposes spectral structure that is masked during steady-state operation. A future system that performs classification on transient operating regimes would offer detection opportunities not available to steady-state-only models.

8.3.4 Progressive Severity and Fault Evolution Modelling

The current severity classifier produces a point-in-time prediction. A future extension could maintain a temporal model of severity evolution, tracking how a given fault develops over weeks or months. Such a model would enable trend-based diagnostics: rather than reporting “Mechanical Looseness, Low severity,” the system could report “Mechanical Looseness, currently Low but progressing toward Medium based on the trajectory over the past three weeks.” This is operationally more useful than a single snapshot and aligns more closely with how maintenance teams reason about asset health.

8.3.5 Live Streaming and Continuous Monitoring

A natural extension of the current batch-file inference path is continuous monitoring, in which vibration data is consumed in real time directly from the sensor rather than from an uploaded recording. The required additions are confined to two layers of the system: a streaming ingestion module that parses the sensor’s wire format and accumulates samples into the same fixed-size windows used by the offline pipeline, and a user-interface module that updates the dashboard’s diagnostic output continuously as new windows arrive. Because the inference engine and the preprocessing pipeline are already stateless with respect to recording boundaries, no modification to the trained models or to the prediction code is required. A smoothing layer that aggregates per-window predictions across a configurable temporal horizon (typically the most recent five to ten seconds) would convert the per-window outputs into a stable rolling diagnosis suitable for operator-facing display.

8.3.6 Operator-Facing UX Refinements

Several user-facing extensions would increase the system’s operational usefulness. Role-specific dashboards, distinct views for on-the-floor operators, maintenance leads, and plant managers, would tailor the information presented to each user’s decision context. Integration with computerized maintenance management systems (CMMS) would push detected faults directly into the existing maintenance ticketing workflow, eliminating the manual step of transferring a diagnosis from the monitoring dashboard into the work-order system. Multi-machine fleet views would let one operator monitor dozens of machines simultaneously, surfacing only the assets that require attention.

8.4 Concluding Remarks

This project demonstrates that high-accuracy automated vibration fault classification and severity estimation can be achieved on commodity hardware with carefully designed preprocessing, principled model selection, and disciplined engineering practices. The two-stage cascaded architecture, the plugin-based model registry, the polished Streamlit application, and the cross-platform packaging

together form a system that is not only a research artifact but a credible foundation for real-world predictive maintenance applications. The honest documentation of limitations, particularly around single-fault training data, bounded speed range, and domain specificity to the test rig, is intended to set the stage for principled extension by future work rather than to advance overstated claims.

The progression from a research prototype to a deployable system has involved choices at every layer of the stack. The choice to train on triaxial rather than uniaxial data improved feature richness. The choice to compute FFT preprocessing offline and standardize per-channel from training data alone preserved the integrity of the held-out evaluation. The choice to cascade fault and severity classification rather than fold both into a single model produced cleaner per-stage learning signals and operationally meaningful outputs. The choice to benchmark seven architectures rather than commit to one produced a defensible justification for the production selection. The choice to pin dependency versions and bundle saved artifacts produced a deployment that operates identically across Windows, macOS, and Linux platforms. Each of these choices, individually small, accumulates into a system that is more robust, more interpretable, and more deployable than the sum of its parts.

The combination of test-set Macro F1 above 0.99 on fault classification, above 0.98 on per-fault severity classification, sub-50-millisecond inference latency on commodity CPU, and reproducible cross-platform deployment positions this work as a credible starting point for industrial predictive maintenance applications. The path from here to deployment on real industrial machinery is documented in the future-work section, and the methodological discipline applied to the work presented in this thesis is, the author hopes, instructive for that next phase.

Bibliography

- [1] Dempster, A., Schmidt, D. F., & Webb, G. I. (2021). *MINIROCKET: A Very Fast (Almost) Deterministic Transform for Time Series Classification*. ACM SIGKDD.
- [2] He, K., Zhang, X., Ren, S., & Sun, J. (2016). *Deep Residual Learning for Image Recognition*. CVPR.
- [3] Randall, R. B. (2011). *Vibration-based Condition Monitoring: Industrial, Aerospace and Automotive Applications*. Wiley.
- [4] Zhao, R., Yan, R., Chen, Z., et al. (2019). *Deep Learning and Its Applications to Machine Health Monitoring*. Mechanical Systems and Signal Processing, 115.

Add additional references as needed.